JavaScript లో LLD (Low-Level Design) - పూర్తి తెలుగు గైడ్

> *ఈ document చదివిన తర్వాత LLD మళ్ళీ మర్చిపోలేవు. ప్రతి principle, ప్రతి design pattern కి real-life scenario, వివరణ, UML ఆలోచన, మరియు run అయ్యే JavaScript code ఉంటాయి. ఇది OOP గైడ్ (`OOPS_Telugu.md`) కి కొనసాగింపు - OOP పునాది అయితే, LLD ఆ పునాది మీద కట్టే భవనం.*

విషయ సూచిక (Table of Contents)

Part 1 — పునాదులు (Foundations)

1. LLD అంటే ఏమిటి? (HLD vs LLD)
2. LLD ని ఎలా approach చేయాలి (Process)
3. Class Relationships (Association, Aggregation, Composition, Dependency)
4. UML Class Diagram చదవడం

Part 2 — Design Principles

5. SOLID (LLD దృష్టితో)
6. DRY, KISS, YAGNI
7. Composition over Inheritance
8. Law of Demeter (కనీస పరిచయ సూత్రం)
9. Program to Interface + Encapsulate What Varies

Part 3 — Creational Patterns (వస్తువులు ఎలా పుట్టాలి)

10. Singleton
11. Factory Method
12. Abstract Factory
13. Builder
14. Prototype

Part 4 — Structural Patterns (వస్తువులు ఎలా కలవాలి)

15. Adapter
16. Bridge
17. Composite
18. Decorator
19. Facade
20. Flyweight
21. Proxy

Part 5 — Behavioral Patterns (వస్తువులు ఎలా మాట్లాడుకోవాలి)

22. Chain of Responsibility
23. Command
24. Iterator
25. Mediator
26. Memento
27. Observer
28. State
29. Strategy
30. Template Method

31. Visitor
32. Interpreter

Part 6 — Real-world Case Studies

33. Parking Lot System
34. LRU Cache
35. Vending Machine (State pattern)
36. Rate Limiter
37. Notification Service (Observer + Strategy)
38. Elevator System
39. Tic-Tac-Toe

Part 7 — Beyond GoF (Practical LLD Patterns)

40. Dependency Injection (DI)
41. Object Pool
42. Null Object
43. Concurrency మరియు Thread-Safety
44. Architecture Patterns (MVC / Layered / Pub-Sub)

Part 8 — Interview & Reference

45. ఏ Pattern ఎప్పుడు వాడాలి? (Cheat Sheet)
46. LLD Interview Framework (5 అడుగులు)
47. Memory Tips Table

Part 1 — పునాదులు (Foundations)

> *LLD నేర్చుకునే ముందు పునాది: LLD అంటే ఏమిటి, entities ఎలా గుర్తించాలి, classes మధ్య సంబంధాలు, UML diagram.*

---

1. LLD అంటే ఏమిటి? (HLD vs LLD)

వివరణ

LLD (Low-Level Design) అంటే ఒక system ని **classes, objects, methods, relationships** స్థాయిలో design చేయడం. అంటే - "ఈ feature ని ఏ classes తో, ఏ methods తో, ఎలా organize చేసి రాయాలి?" అనే ప్రశ్నకి సమాధానం.

- **HLD (High-Level Design)** = పెద్ద బొమ్మ. Services, databases, load balancers, APIs - system architecture.
- **LLD (Low-Level Design)** = ఒక్కో component లోపల class-level blueprint.

Real-life Scenario

> *ఒక ఇల్లు కడుతున్నామనుకో:*
>
> - ***HLD** = ఎన్ని గదులు, ఎక్కడ kitchen, ఎక్కడ bathroom - master plan (architect గీసేది).*
> - ***LLD** = ఒక్కో గోడకి ఏ ఇటుకలు, switch board ఎక్కడ, wiring ఎలా - detailed engineering drawing.*

HLD vs LLD పోలిక

అంశం	HLD	LLD
Level	System / Architecture	Class / Object
ప్రశ్న	ఏ components అవసరం?	ఆ component లోపల ఏ classes?
Output	Architecture diagram	Class diagram, code structure
ఉదా	"Payment service + DB + queue"	"Payment, UPI, Card classes + Strategy pattern"
Audience	Architects, teams	Developers

LLD ఎందుకు ముఖ్యం?

1. **Maintainable** - రేపు మార్పు చేయాలంటే ఒకే చోట మార్చగలగాలి
2. **Extensible** - కొత్త feature add చేయాలంటే పాత code విరగకూడదు (Open/Closed)
3. **Reusable** - ఒకసారి రాసిన class మళ్ళీ మళ్ళీ వాడగలగాలి
4. **Testable** - చిన్న, స్పష్టమైన classes ని easy గా test చేయవచ్చు
5. **Interviews** - product companies LLD round తప్పకుండా అడుగుతాయి

> ***గుర్తుంచుకో:*** *OOP = tools (class, object, inheritance...). LLD = ఆ tools ని ఎప్పుడు, ఎలా వాడాలో నేర్పే craft.*

---

2. LLD ని ఎలా approach చేయాలి (Process)

వివరణ

ఒక problem ("Design a Parking Lot") ఇచ్చినప్పుడు నేరుగా code రాయకూడదు. ఒక క్రమం (process) follow చేయాలి.

6 అడుగుల Process

అడుగు	ఏం చేయాలి	ఉదా (Parking Lot)
1. Requirements	ఏం కావాలో clarify చేయి	ఎన్ని floors? bike + car?
2. Entities గుర్తించు	Nouns → classes	ParkingLot, Slot, Vehicle, Ticket
3. Attributes + Methods	ప్రతి class కి data + behaviour	Slot: id, isFree, park()
4. Relationships	classes మధ్య సంబంధం	Lot **has** Floors, Floor **has** Slots
5. Design Patterns	ఎక్కడ ఏ pattern సరిపోతుంది	Strategy (fee), Factory (vehicle)
6. Code + Refine	classes రాసి, SOLID check చేయి	test చేసి improve చేయి

Nouns → Classes, Verbs → Methods (Trick)

> *Requirement వాక్యాలలో **naamavaachakaalu (nouns)** classes అవుతాయి, **kriyalu (verbs)** methods అవుతాయి.*
>
> *"A **user books** a **ticket** for a **show**"*
>
> - *Nouns →* `User` *,* `Ticket` *,* `Show` *(classes)*
> - *Verb →* `book()` *(method)*

Key Points

- ముందు మాట్లాడు (requirements), తర్వాత గీయి (diagram), చివరిలో రాయి (code)
- Over-engineering వద్దు - అవసరం లేని patterns కూరకు (YAGNI - Topic 6)
- ఒక్కో class కి ఒకే బాధ్యత (SRP) - ఇది LLD యొక్క గుండె

---

3. Class Relationships (వస్తువుల మధ్య సంబంధాలు)

వివరణ

LLD లో అతి ముఖ్యమైనది - classes మధ్య **సంబంధాలు** సరిగ్గా గుర్తించడం. 4 ముఖ్య సంబంధాలు ఉన్నాయి. వీటిని strength (బలం) క్రమంలో గుర్తుంచుకో.

Relationship	అర్థం	జీవితకాలం (Lifetime)	ఉదా
Association	"uses-a" / తెలుసు	స్వతంత్రం	Teacher — Student
Aggregation	"has-a" (weak)	విడిపోగలవు	Team — Player
Composition	"has-a" (strong) / part-of	కలిసి బతుకుతాయి, కలిసి చస్తాయి	House — Room
Dependency	"depends-on" (తాత్కాలికం)	క్షణికం	Order — PaymentService

Real-life Scenario

> - ***Aggregation:*** *Cricket **Team** కి **Players** ఉంటారు. Team రద్దయినా players బతికే ఉంటారు (వేరే team కి వెళ్తారు). Weak bond.*
> - ***Composition:*** ***ఇల్లు** కి **గదులు** ఉంటాయి. ఇల్లు కూలిపోతే గదులు కూడా పోతాయి. Strong bond - గది ఇల్లు లేకుండా ఒంటరిగా ఉండదు.*

Code

```
// ---------- Association: రెండూ స్వతంత్రం, ఒకరికొకరు తెలుసు ----------
class Student {
  constructor(name) {
    this.name = name;
  }
}
class Teacher {
  teach(student) {
    // Teacher, Student ని "use" చేస్తోంది కానీ own చేయడం లేదు
    console.log(`Teacher ${student.name} కి పాఠం చెప్తోంది.`);
  }
}

// ---------- Aggregation: Team "has" Players (weak - విడిపోగలవు) ----------
class Player {
  constructor(name) {
    this.name = name;
  }
```

```
}
class Team {
  constructor(name) {
    this.name = name;
    this.players = []; // players బయట create అవుతారు, ఇక్కడ కేవలం hold చేస్తాం
  }
  addPlayer(player) {
    this.players.push(player);
  }
}

// ---------- Composition: House "owns" Rooms (strong – కలిసి పుడతాయి/చస్తాయి) --------
--
class Room {
  constructor(type) {
    this.type = type;
  }
}
class House {
  constructor() {
    // Rooms ని House లోపలే create చేస్తాం – House పోతే Rooms కూడా పోతాయి
    this.rooms = [new Room("Kitchen"), new Room("Bedroom")];
  }
}

// ---------- Dependency: Order "depends on" PaymentService (తాత్కాలికం) ----------
class PaymentService {
  pay(amount) {
    console.log(`Rs.${amount} paid.`);
  }
}
class Order {
  checkout(amount) {
    const payment = new PaymentService(); // క్షణికంగా వాడి వదిలేస్తాం
    payment.pay(amount);
  }
}

// Testing
const teacher = new Teacher();
teacher.teach(new Student("Yaswanth")); // Teacher Yaswanth కి పాఠం చెప్తోంది.

const team = new Team("Chargers");
team.addPlayer(new Player("Kohli"));
console.log(`${team.name} team లో ${team.players.length} player(s).`); // 1 player

const house = new House();
console.log(`House లో ${house.rooms.length} rooms (composition).`); // 2 rooms

new Order().checkout(500); // Rs.500 paid.
```

Association vs Aggregation vs Composition (UML గుర్తులు)

```
Association:  Teacher ──────────── Student      (సాదా line)
Aggregation:  Team    ◇─────────── Player       (ఖాళీ వజ్రం - weak)
Composition:  House   ◆─────────── Room         (నిండు వజ్రం - strong)
Dependency:   Order   ----------> PaymentService (చుక్కల బాణం)
```

> ***గుర్తుంచుకో:*** *ఖాళీ వజ్రం (◇) = Aggregation (part విడిగా బతుకుతుంది). నిండు వజ్రం (◆) = Composition (part owner తో పాటే పోతుంది).*

---

4. UML Class Diagram చదవడం

వివరణ

LLD interviews లో మాటలతో కాకుండా **class diagram** గీయాలి. UML class box లో 3 అరలు ఉంటాయి: పేరు, attributes, methods.

ఒక UML Class Box

```
┌──────────────────────────┐
│       BankAccount        │   <- Class పేరు
├──────────────────────────┤
│ - accountNumber: string  │   <- Attributes
│ - balance: number        │      (- private, + public, # protected)
├──────────────────────────┤
│ + deposit(amt): void     │   <- Methods
│ + withdraw(amt): boolean │
│ - validate(): boolean    │
└──────────────────────────┘
```

Visibility గుర్తులు

గుర్తు	అర్థం	JavaScript
+	public	`this.x`
-	private	`#x`
#	protected	`_x` (convention)
_ (underline)	static	`static x`

Relationship Arrows (బాణాలు)

బాణం	సంబంధం
──▷ (ఖాళీ triangle)	Inheritance (extends)
----▷ (చుక్కల + triangle)	Interface implement (realization)
───>	Association

◇——	Aggregation
◆——	Composition
---->	Dependency

Key Points

- Box లో 3 అరలు: name / attributes / methods - ఎప్పుడూ ఈ క్రమంలో
- `-` private, `+` public - visibility ని బాణాల్లా ముఖ్యంగా చూపించు
- Interview లో perfect diagram కంటే స్పష్టమైన diagram ముఖ్యం
- Multiplicity రాయవచ్చు: `Team "1" ◇—— "*" Player` (ఒక team కి చాలా players)

Sequence Diagram (కాలక్రమంలో objects మధ్య calls)

> *Class diagram = **structure** (ఏ classes ఉన్నాయి). Sequence diagram = **behaviour over time** (ఏ order లో methods call అవుతాయి). Interview లో "flow చూపించు" అంటే ఇది గీయాలి.*

```
User        Controller      Service       Database
 |              |              |              |
 | login(u,p)   |              |              |
 |─────────────>|              |              |
 |              | validate(u,p)|              |
 |              |─────────────>|              |
 |              |              | findUser(u)  |
 |              |              |─────────────>|
 |              |              |<─ ─ ─ ─ ─ ─ ─|  (user row)
 |              |<─ ─ ─ ─ ─ ─ ─|  (true)      |
 |<─ ─ ─ ─ ─ ─ ─|  (token)     |              |
 |              |              |              |
```

- నిలువు గీత (|) = ఒక్కో object యొక్క lifeline (కాలం కిందికి సాగుతుంది)
- నిండు బాణం (——>) = method call; చుక్కల బాణం (--- ---) = return value
- Login, payment, booking లాంటి **flows** ని చూపించడానికి perfect

---

Part 2 — Design Principles

> *Patterns కి ముందు principles. ఇవి ఏ pattern వాడినా వర్తించే సార్వత్రిక సూత్రాలు - మంచి design కి దిక్సూచి.*

---

5. SOLID (LLD దృష్టితో)

వివరణ

SOLID = మంచి LLD కి 5 మూల సూత్రాలు. (వివరమైన Telugu examples కోసం `OOPS_Telugu.md` Topic 32 చూడు.) ఇక్కడ LLD problem లో వీటిని ఎలా వాడాలో చూద్దాం.

అక్షరం	సూత్రం	LLD లో అర్థం
S	Single Responsibility	ఒక class మారడానికి ఒకే కారణం ఉండాలి
O	Open/Closed	కొత్త feature = కొత్త class, పాత code touch వద్దు

L	Liskov Substitution	Child ని parent స్థానంలో పెట్టినా break అవ్వకూడదు
I	Interface Segregation	పెద్ద interface కాదు, చిన్న role-based interfaces
D	Dependency Inversion	Concrete కాదు, abstraction మీద depend అవ్వు

Code - SRP + OCP + DIP కలిసి ఒక ఉదాహరణలో

```
// D — Notification concrete channel మీద కాదు, "send()" contract మీద depend
class SmsChannel {
  send(msg) {
    console.log(`SMS: ${msg}`);
  }
}
class EmailChannel {
  send(msg) {
    console.log(`Email: ${msg}`);
  }
}

// O — కొత్త channel (WhatsApp) add చేయాలంటే ఈ class లు మార్చక్కర్లేదు
class WhatsAppChannel {
  send(msg) {
    console.log(`WhatsApp: ${msg}`);
  }
}

// S — Notifier కి ఒకే బాధ్యత: message పంపడం (channel ఎంపిక దాని పని కాదు)
class Notifier {
  constructor(channel) {
    this.channel = channel;
  } // dependency injection
  notify(msg) {
    this.channel.send(msg);
  }
}

new Notifier(new SmsChannel()).notify("OTP 1234"); // SMS: OTP 1234
new Notifier(new EmailChannel()).notify("Welcome!"); // Email: Welcome!
new Notifier(new WhatsAppChannel()).notify("Order shipped"); // WhatsApp: Order
shipped
```

Key Points

- SRP - "ఈ class ఎన్ని కారణాలకి మారుతుంది?" ఒకటి కంటే ఎక్కువ ఉంటే విడగొట్టు
- OCP - `if/else / switch` పెరుగుతూ ఉంటే అది polymorphism అడుగుతోంది
- DIP - `new` ని class లోపల కాకుండా బయట చేసి inject చేయి (testable అవుతుంది)

---

6. DRY, KISS, YAGNI

వివరణ

SOLID తో పాటు రోజువారీ 3 practical మంత్రాలు.

సూత్రం	పూర్తి రూపం	అర్థం
DRY	Don't Repeat Yourself	ఒకే logic రెండుచోట్ల ఉంటే ఒక చోటికి తీసుకురా
KISS	Keep It Simple, Stupid	సాధ్యమైనంత simple గా ఉంచు
YAGNI	You Aren't Gonna Need It	ఇప్పుడు అవసరం లేనిది ఇప్పుడు రాయకు

Real-life Scenario

> - ***DRY:*** *ప్రతి function లో tax లెక్క copy-paste చేయకుండా ఒక* `calculateTax()` *రాయి.*
> - ***KISS:*** *ఒక్క line తో అయ్యే పనికి 3 design patterns తేవద్దు.*
> - ***YAGNI:*** *"రేపు 10 currencies రావచ్చు" అని ఇప్పుడే multi-currency engine రాయకు - వచ్చినప్పుడు చూద్దాం.*

Code - DRY ఉదాహరణ

```
// WRONG (repetition): ప్రతి చోట discount logic copy
// function priceA(p) { return p - p * 0.1; }
// function priceB(p) { return p - p * 0.1; }

// DRY: ఒకే చోట logic
function applyDiscount(price, percent) {
  return price - (price * percent) / 100;
}

console.log(applyDiscount(1000, 10)); // 900
console.log(applyDiscount(2000, 25)); // 1500

// KISS: సూటిగా - unnecessary abstraction లేదు
function isAdult(age) {
  return age >= 18;
}
console.log(isAdult(20)); // true
```

Key Points

- DRY vs విపరీతం: ఒకే *coincidence* ని force గా merge చేయకు (rule of three - 3 సార్లు repeat అయితేనే extract చేయి)
- KISS - "clever" code కంటే "clear" code మంచిది
- YAGNI - future కోసం over-design = wasted effort + bugs

---

7. Composition over Inheritance

వివరణ

Inheritance ("is-a") శక్తివంతమైనది కానీ గట్టిగా bind చేస్తుంది (tight coupling). చాలా సందర్భాల్లో **Composition** ("has-a") flexible గా ఉంటుంది - behaviours ని విడి parts గా కలుపుతాం.

Real-life Scenario

> ***Problem:*** *`Robot` ఒక `Animal` కాదు, కానీ దానికి walk చేయగలగాలి. `Animal` నుండి inherit చేస్తే eat(), sleep() కూడా వస్తాయి - అవసరం లేనివి. బదులుగా `walk` అనే behaviour ని కలుపుతాం (compose).*

Code

```
// Behaviours ని విడి, reusable objects గా చేయడం
const canWalk = (state) => ({
  walk: () => console.log(`${state.name} నడుస్తోంది.`),
});
const canFly = (state) => ({
  fly: () => console.log(`${state.name} ఎగురుతోంది.`),
});
const canSwim = (state) => ({
  swim: () => console.log(`${state.name} ఈదుతోంది.`),
});

// అవసరమైన behaviours ని మాత్రమే కలిపి object తయారు చేయడం (compose)
function createDuck(name) {
  const state = { name };
  return Object.assign({}, canWalk(state), canFly(state), canSwim(state));
}
function createRobot(name) {
  const state = { name };
  return Object.assign({}, canWalk(state)); // Robot కి walk మాత్రమే
}

const duck = createDuck("Donald");
duck.walk(); // Donald నడుస్తోంది.
duck.fly(); // Donald ఎగురుతోంది.
duck.swim(); // Donald ఈదుతోంది.

const robot = createRobot("R2D2");
robot.walk(); // R2D2 నడుస్తోంది.
// robot.fly — లేదు (అవసరం లేని behaviour రాలేదు)
console.log(typeof robot.fly); // undefined
```

ఎప్పుడు ఏది?

పరిస్థితి	ఎంపిక
నిజమైన "is-a" + stable hierarchy	Inheritance
Behaviours mix-and-match కావాలి	**Composition**
Runtime లో behaviour మారాలి	**Composition**
Deep inheritance tree వస్తోంది (3+ levels)	**Composition** వైపు మొగ్గు

> ***Gang of Four సూత్రం:*** *"Favour object composition over class inheritance."*

---

8. Law of Demeter (కనీస పరిచయ సూత్రం)

వివరణ

Law of Demeter (LoD) = "నీ friends తో మాత్రమే మాట్లాడు, friends యొక్క friends తో కాదు." ఒక method కేవలం దగ్గరి objects తోనే interact అవ్వాలి - deep గా `a.b.c.d` chain చేయకూడదు.

Real-life Scenario

> *Hotel లో బిల్లు కట్టేటప్పుడు నువ్వు నీ **wallet** ని waiter కి ఇవ్వవు, డబ్బు నువ్వే తీసి ఇస్తావు. అంటే waiter `customer.wallet.cash.count()` చేయకూడదు - `customer.pay(amount)` అడగాలి.*

Code

```
// WRONG - Train wreck: wallet లోపలికి చొరబడటం (a.b.c.d)
class Wallet {
  constructor(cash) {
    this.cash = cash;
  }
}
class CustomerBad {
  constructor() {
    this.wallet = new Wallet(1000);
  }
}
// waiter చేసేది: customer.wallet.cash -- LoD violation!

// CORRECT - Customer తనే pay() ఇస్తుంది; wallet బయటికి కనిపించదు
class CustomerGood {
  #wallet;
  constructor(cash) {
    this.#wallet = new Wallet(cash);
  }

  pay(amount) {
    if (this.#wallet.cash >= amount) {
      this.#wallet.cash -= amount;
      console.log(`Rs.${amount} paid. Balance: Rs.${this.#wallet.cash}`);
      return amount;
    }
    console.log("Insufficient cash!");
    return 0;
  }
}

class Waiter {
  collectBill(customer, amount) {
    // wallet లోపల ఏముందో waiter కి తెలియదు - కేవలం pay() అడుగుతాడు
    return customer.pay(amount);
```

```
  }
}

const customer = new CustomerGood(1000);
new Waiter().collectBill(customer, 300); // Rs.300 paid. Balance: Rs.700
```

Key Points

- ఒక method లో `.` ఒకటి కంటే ఎక్కువ chain (`a.getB().getC()`) కనిపిస్తే LoD alarm
- లోపలి structure ని బయటికి leak చేయకు → encapsulation బలపడుతుంది
- ఫలితం: ఒక class మారితే మిగతావి తక్కువ affect అవుతాయి (loose coupling)

---

9. Program to Interface + Encapsulate What Varies

వివరణ

రెండు బంగారు LLD సూత్రాలు:

1. **Program to an interface, not an implementation** - concrete class కాదు, contract (abstraction) మీద ఆధారపడు.
2. **Encapsulate what varies** - మారే భాగాన్ని విడిగా తీసి దాచు; మిగతా code stable గా ఉంటుంది.

Real-life Scenario

> ***Power socket*** *ఒక interface. TV, phone charger, fridge - ఏదైనా ఆ socket కి plug అవుతాయి. Socket కి లోపల ఏ device ఉందో పట్టదు - అది "plug shape" అనే contract మీద మాత్రమే ఆధారపడుతుంది.*

Code

```
// "Interface" (JS లో contract) - అన్ని sorters area() లాంటి compare() ఇవ్వాలి
// మారేది = sorting strategy. దాన్ని encapsulate చేస్తాం.
class BubbleSort {
  sort(arr) {
    console.log("Bubble sort వాడుతోంది");
    return [...arr].sort((a, b) => a - b);
  }
}
class QuickSort {
  sort(arr) {
    console.log("Quick sort వాడుతోంది");
    return [...arr].sort((a, b) => a - b);
  }
}

// Processor ఏ concrete sorter అనేది తెలుసుకోదు - "sort()" contract మీద మాత్రమే
ఆధారపడుతుంది
class DataProcessor {
  constructor(sorter) {
    this.sorter = sorter;
  } // program to interface
  process(data) {
    return this.sorter.sort(data);
```

```
  }
}

const data = [5, 2, 9, 1];
console.log(new DataProcessor(new BubbleSort()).process(data)); // [1,2,5,9]
console.log(new DataProcessor(new QuickSort()).process(data)); // [1,2,5,9]
// రేపు MergeSort వచ్చినా DataProcessor మార్చక్కర్లేదు!
```

Key Points

- Concrete class పేరు code అంతటా చిమ్మకు - ఒక చోట (factory/injection) మాత్రమే వాడు
- "ఏది మారుతోంది?" అని అడుగు → అదే part ని విడి class/strategy గా తీయి
- ఇదే Strategy pattern (Topic 29) కి పునాది

Part 3 — Creational Patterns

> *Creational patterns = **objects ఎలా పుట్టాలి** అనేది control చేసేవి. `new` ని నేరుగా చిమ్మకుండా, creation logic ని ఒక చోట దాచుతాయి.*

---

10. Singleton

వివరణ

Singleton = ఒక class నుండి **ఒకే ఒక్క object** మాత్రమే ఉండాలి, అది అందరికీ share అవ్వాలి. (వివరమైన version కోసం `OOPS_Telugu.md` Topic 25 చూడు.)

Real-life Scenario

> *ఒక కంపెనీకి **ఒకే CEO**. ఎవరు అడిగినా అదే CEO. కొత్త CEO ని ప్రతిసారి create చేయరు.*
> *Config, Logger, DB connection pool - వీటికి Singleton సరిపోతుంది.*

Code

```
class AppConfig {
  static #instance = null;

  constructor() {
    if (AppConfig.#instance) return AppConfig.#instance; // ఉన్నదాన్నే తిప్పి ఇవ్వడం
    this.settings = { theme: "dark", lang: "te" };
    AppConfig.#instance = this;
  }

  static getInstance() {
    if (!AppConfig.#instance) AppConfig.#instance = new AppConfig();
    return AppConfig.#instance;
  }
}

const c1 = AppConfig.getInstance();
const c2 = AppConfig.getInstance();
```

```
console.log(c1 === c2); // true — ఒకటే object
c1.settings.theme = "light";
console.log(c2.settings.theme); // light — c1, c2 ఒకటే కాబట్టి
```

ఎప్పుడు వాడాలి / వద్దు

వాడు	వద్దు
Logger, Config, Cache	ప్రతిచోటా global state కోసం (test కష్టం)
ఒకే resource (DB pool)	అనవసరంగా - inject చేయడం మంచిది

> ***జాగ్రత్త:*** *Singleton అతిగా వాడితే hidden global state వస్తుంది → testing కష్టం. అవసరమైనప్పుడే.*

---

11. Factory Method

వివరణ

Factory Method = object ని `new` తో నేరుగా కాకుండా, ఒక **factory method** ద్వారా create చేయడం. ఏ class object రావాలో ఆ method నిర్ణయిస్తుంది.

Real-life Scenario

> ***Restaurant*** *లో నువ్వు kitchen లోకి వెళ్ళి వంట చేయవు. "One pizza" అని* ***order*** *ఇస్తావు - kitchen (factory) సరైన dish తయారు చేసి ఇస్తుంది. నీకు internal recipe అక్కర్లేదు.*

Code

```
class Circle {
  draw() {
    console.log("వృత్తం గీస్తున్నా ⭕");
  }
}
class Square {
  draw() {
    console.log("చతురస్రం గీస్తున్నా ⬜");
  }
}
class Triangle {
  draw() {
    console.log("త్రిభుజం గీస్తున్నా 🔺");
  }
}

// Factory — ఏ shape కావాలో string బట్టి సరైన object ఇస్తుంది
class ShapeFactory {
  static create(type) {
    switch (type) {
      case "circle":
        return new Circle();
      case "square":
```

```
        return new Square();
      case "triangle":
        return new Triangle();
      default:
        throw new Error(`Unknown shape: ${type}`);
    }
  }
}

// Client కి concrete class పేర్లు అక్కర్లేదు — "circle" అంటే చాలు
["circle", "square", "triangle"].forEach((t) => {
  ShapeFactory.create(t).draw();
});
// వృత్తం గీస్తున్నా ⭕
// చతురస్రం గీస్తున్నా ⬜
// త్రిభుజం గీస్తున్నా 🔺
```

Key Points

- Creation logic ఒకే చోట (factory) - కొత్త type add చేస్తే client code మారదు
- Client concrete classes మీద కాకుండా factory + common contract మీద ఆధారపడుతుంది
- `switch` పెరిగితే registry (map) వాడి OCP-friendly చేయవచ్చు

---

12. Abstract Factory

వివరణ

Abstract Factory = "factories యొక్క factory". సంబంధిత objects యొక్క **కుటుంబాన్ని (family)** ఒకేసారి create చేస్తుంది - అవి ఒకదానితో ఒకటి match అవ్వేలా.

Real-life Scenario

> ***Furniture showroom** లో "Victorian style" అడిగితే - Victorian chair + Victorian table + Victorian sofa అన్నీ ఒకే style లో వస్తాయి. "Modern" అడిగితే మొత్తం modern family. Styles mix అవ్వవు.*

Code

```
// Product family 1: Light theme UI
class LightButton {
  render() {
    console.log("తెల్ల button ☀️");
  }
}
class LightCheckbox {
  render() {
    console.log("తెల్ల checkbox ☀️");
  }
}

// Product family 2: Dark theme UI
class DarkButton {
```

```
  render() {
    console.log("నల్ల button 🌙");
  }
}
class DarkCheckbox {
  render() {
    console.log("నల్ల checkbox 🌙");
  }
}

// Abstract Factories – ఒక్కో family కి ఒక factory
class LightThemeFactory {
  createButton() {
    return new LightButton();
  }
  createCheckbox() {
    return new LightCheckbox();
  }
}
class DarkThemeFactory {
  createButton() {
    return new DarkButton();
  }
  createCheckbox() {
    return new DarkCheckbox();
  }
}

// Client ఒక factory తీసుకొని పూర్తి matching UI తయారు చేస్తుంది
function buildUI(factory) {
  factory.createButton().render();
  factory.createCheckbox().render();
}

console.log("--- User dark mode ఎంచుకున్నాడు ---");
buildUI(new DarkThemeFactory()); // నల్ల button + నల్ల checkbox (అన్నీ match)
console.log("--- User light mode ఎంచుకున్నాడు ---");
buildUI(new LightThemeFactory()); // తెల్ల button + తెల్ల checkbox
```

Factory Method vs Abstract Factory

	Factory Method	Abstract Factory
ఏం create చేస్తుంది	ఒక్క product	Product family (చాలా related products)
ఉదా	`create("circle")`	Dark theme మొత్తం UI
ముఖ్యం	ఒక object type ఎంపిక	Products అన్నీ match అవ్వడం

---

13. Builder

వివరణ

Builder = చాలా parameters ఉన్న object ని **అడుగడుగునా (step by step)** తయారు చేయడం. Constructor లో 10 arguments పెట్టే బదులు, readable method chaining వాడతాం.

Real-life Scenario

> ***Subway*** *sandwich: bread ఎంచుకో → veggies → sauce → toast? ఒక్కో step జోడించి చివరిలో sandwich ready. అన్నీ ఒకేసారి చెప్పక్కర్లేదు, optional వి skip చేయవచ్చు.*

Code

```
class Burger {
  constructor(builder) {
    this.size = builder.size;
    this.cheese = builder.cheese;
    this.patty = builder.patty;
    this.veggies = builder.veggies;
  }
  describe() {
    const parts = [`${this.size} burger`];
    if (this.patty) parts.push("patty");
    if (this.cheese) parts.push("cheese");
    if (this.veggies) parts.push("veggies");
    console.log(parts.join(" + "));
  }
}

class BurgerBuilder {
  constructor(size) {
    this.size = size;
  } // size తప్పనిసరి
  addCheese() {
    this.cheese = true;
    return this;
  } // this return → chaining
  addPatty() {
    this.patty = true;
    return this;
  }
  addVeggies() {
    this.veggies = true;
    return this;
  }
  build() {
    return new Burger(this);
  }
}

// Method chaining తో అడుగడుగునా build — optional వి skip చేయవచ్చు
const burger = new BurgerBuilder("Large")
```

```
  .addPatty()
  .addCheese()
  .addVeggies()
  .build();
burger.describe(); // Large burger + patty + cheese + veggies

const plain = new BurgerBuilder("Small").addPatty().build();
plain.describe(); // Small burger + patty
```

Key Points

- చాలా (ముఖ్యంగా optional) parameters ఉంటే Builder - "telescoping constructor" నొప్పి పోతుంది
- ప్రతి method `this` return చేస్తే fluent chaining (Method Chaining - OOP Topic 16)
- Immutable object తయారీకి కూడా బాగుంటుంది (build() తర్వాత మార్చలేం)

---

14. Prototype

వివరణ

Prototype = ఒక object ని scratch నుండి కాకుండా, ఉన్న object ని **clone** చేసి కొత్తది తయారు చేయడం. Creation ఖరీదైనప్పుడు (heavy setup) ఉపయోగం.

Real-life Scenario

> *ఒక **resume template** ని ప్రతిసారి మొదటి నుండి type చేయవు - ఉన్నదాన్ని **copy** చేసి పేరు, details మారుస్తావు. అదే prototype cloning.*

Code

```
class Enemy {
  constructor(type, health, weapon) {
    this.type = type;
    this.health = health;
    this.weapon = weapon;
  }
  // clone - ఉన్న object నుండి కొత్తది (deep-ish copy)
  clone() {
    return new Enemy(this.type, this.health, this.weapon);
  }
  describe() {
    console.log(`${this.type} (HP:${this.health}, weapon:${this.weapon})`);
  }
}

// ఖరీదైన base enemy ఒకసారి తయారు చేసి, మిగతా వాటిని clone చేస్తాం
const baseOrc = new Enemy("Orc", 100, "Axe");

const orc1 = baseOrc.clone();
const orc2 = baseOrc.clone();
orc2.weapon = "Sword"; // clone ని స్వతంత్రంగా మార్చవచ్చు
```

```
baseOrc.describe(); // Orc (HP:100, weapon:Axe)
orc1.describe(); // Orc (HP:100, weapon:Axe)
orc2.describe(); // Orc (HP:100, weapon:Sword) — base మారలేదు

// JS native: Object.create / structuredClone కూడా prototype ఆలోచనే
const config = { retries: 3, nested: { debug: true } };
const clonedConfig = structuredClone(config); // deep clone
clonedConfig.nested.debug = false;
console.log(config.nested.debug); // true — original safe
console.log(clonedConfig.nested.debug); // false
```

Key Points

- Object creation ఖరీదైనప్పుడు (DB hit, heavy compute) clone వేగంగా
- **Shallow vs Deep** clone జాగ్రత్త: nested objects shared అవ్వకుండా `structuredClone` వాడు
- JavaScript prototype chain (OOP Topic 13) ఈ pattern యొక్క native రూపమే

Part 4 — Structural Patterns

> *Structural patterns = **objects/classes ని ఎలా కలిపి పెద్ద structure** తయారు చేయాలో చెప్పేవి. Objects మధ్య సంబంధాలను simple గా ఉంచుతాయి.*

---

15. Adapter

వివరణ

Adapter = రెండు incompatible interfaces మధ్య **అనువాదకుడు (translator)**. ఒక class ఆశించే shape కి, వేరే class ని సరిపడేలా చేస్తుంది.

Real-life Scenario

> *నీ laptop కి **US plug**, ఇంట్లో **Indian socket**. మధ్యలో **travel adapter** పెడతావు - రెండూ మారవు, adapter అనుసంధానం చేస్తుంది.*

Code

```
// మన app ఆశించే interface: log(message)
class ModernLogger {
  log(message) {
    console.log(`[LOG] ${message}`);
  }
}

// పాత 3rd-party library — వేరే method పేరు (writeToFile)
class OldLibrary {
  writeToFile(text) {
    console.log(`పాత library file కి రాస్తోంది: "${text}"`);
  }
}
```

```
// Adapter - OldLibrary ని ModernLogger interface లా కనిపించేలా చేస్తుంది
class LoggerAdapter {
  constructor(oldLib) {
    this.oldLib = oldLib;
  }
  log(message) {
    this.oldLib.writeToFile(message);
  } // అనువాదం
}

// Client కేవలం log() మాత్రమే వాడతాడు - లోపల ఏ library అనేది పట్టదు
function runApp(logger) {
  logger.log("App started");
}

runApp(new ModernLogger()); // [LOG] App started
runApp(new LoggerAdapter(new OldLibrary())); // పాత library file కి రాస్తోంది: "App
started"
```

Key Points

- పాత/3rd-party code ని మార్చకుండా కొత్త system కి కలపడానికి
- Adapter "wrap" చేస్తుంది; interface translate చేస్తుంది (behaviour మార్చదు)

---

16. Bridge

వివరణ

Bridge = రెండు స్వతంత్రంగా మారే dimensions ని విడదీసి (abstraction vs implementation), వాటిని ఒక "bridge" (composition) తో కలపడం. Class explosion ని ఆపుతుంది.

Real-life Scenario

> ***Remote*** *(abstraction) మరియు* ***Device*** *(TV/Radio - implementation) విడివిడిగా. ఏ remote అయినా ఏ device తోనైనా పని చేస్తుంది. లేకపోతే TVRemote, RadioRemote, SmartTVRemote... explosion అవుతుంది.*

Code

```
// Implementation side - devices
class TV {
  turnOn() {
    console.log("TV ఆన్ అయింది 📺");
  }
  setVolume(v) {
    console.log(`TV volume: ${v}`);
  }
}
class Radio {
  turnOn() {
    console.log("Radio ఆన్ అయింది 📻");
  }
```

```
  setVolume(v) {
    console.log(`Radio volume: ${v}`);
  }
}

// Abstraction side – remotes (device ని compose చేస్తాయి = bridge)
class RemoteControl {
  constructor(device) {
    this.device = device;
  }
  power() {
    this.device.turnOn();
  }
}
// Abstraction ని విడిగా పెంచవచ్చు – devices మార్చకుండా
class AdvancedRemote extends RemoteControl {
  mute() {
    this.device.setVolume(0);
    console.log("Muted 🔇");
  }
}

// ఏ remote + ఏ device అయినా mix చేయవచ్చు (2+2 combos, 4 classes కాదు)
new RemoteControl(new TV()).power(); // TV ఆన్ అయింది 📺
const r = new AdvancedRemote(new Radio());
r.power(); // Radio ఆన్ అయింది 📻
r.mute(); // Radio volume: 0 / Muted 🔇
```

Key Points

- రెండు dimensions స్వతంత్రంగా మారుతున్నప్పుడు (remote రకం × device రకం)
- Inheritance explosion (M×N classes) → composition (M+N classes)

---

17. Composite

వివరణ

Composite = వ్యక్తిగత objects (leaf) మరియు objects గుంపులని (composite) **ఒకేలా** treat చేయడం. Tree structure - part-whole hierarchy.

Real-life Scenario

> ***Folder** లో files ఉంటాయి, మరో folders ఉంటాయి. "size ఎంత?" అని folder ని అడిగితే, అది లోపలి అన్నిటినీ కలిపి చెప్తుంది. File అయినా folder అయినా `getSize()` ఒకేలా అడుగుతాం.*

Code

```
// Leaf – వ్యక్తిగత file
class FileItem {
  constructor(name, size) {
    this.name = name;
```

```
    this.size = size;
  }
  getSize() {
    return this.size;
  }
}

// Composite – folder (files + folders రెండూ కలిగి ఉంటుంది)
class Folder {
  constructor(name) {
    this.name = name;
    this.children = [];
  }
  add(item) {
    this.children.push(item);
    return this;
  }
  // File అయినా Folder అయినా అందరికీ getSize() ఉంది – ఒకేలా treat
  getSize() {
    return this.children.reduce((sum, child) => sum + child.getSize(), 0);
  }
}

const root = new Folder("root");
root.add(new FileItem("a.txt", 100)).add(new FileItem("b.txt", 200));

const sub = new Folder("images");
sub.add(new FileItem("pic.png", 500));
root.add(sub);

console.log(`Total size: ${root.getSize()} KB`); // 800 KB (100+200+500, recursive)
console.log(`Images size: ${sub.getSize()} KB`); // 500 KB
```

Key Points

- Leaf మరియు Composite ఒకే interface (`getSize`) పంచుకుంటాయి → client తేడా చూడక్కర్లేదు
- Recursion సహజంగా వస్తుంది (folder లోపల folder)
- UI trees, org charts, file systems, menus - అన్నీ Composite

---

18. Decorator

వివరణ

Decorator = ఒక object కి **runtime లో** కొత్త behaviour ని పొరలుగా (layers) చుట్టడం - subclass లు తయారు చేయకుండా.

Real-life Scenario

> ***Coffee**: base coffee → milk జోడించు → sugar జోడించు → cream జోడించు. ప్రతి addition price + description ని పెంచుతుంది. MilkSugarCreamCoffee అనే class వద్దు - పొరలుగా చుడతాం.*

Code

```
// Base component
class Coffee {
  cost() {
    return 50;
  }
  description() {
    return "Coffee";
  }
}

// Decorators — ఒక coffee ని wrap చేసి కొత్తది జోడిస్తాయి
class MilkDecorator {
  constructor(coffee) {
    this.coffee = coffee;
  }
  cost() {
    return this.coffee.cost() + 10;
  }
  description() {
    return this.coffee.description() + " + Milk";
  }
}
class SugarDecorator {
  constructor(coffee) {
    this.coffee = coffee;
  }
  cost() {
    return this.coffee.cost() + 5;
  }
  description() {
    return this.coffee.description() + " + Sugar";
  }
}

// పొరలుగా చుట్టడం (wrap) — ఎన్ని అయినా, ఏ క్రమంలోనైనా
let order = new Coffee();
order = new MilkDecorator(order);
order = new SugarDecorator(order);
order = new SugarDecorator(order); // డబుల్ sugar!

console.log(order.description()); // Coffee + Milk + Sugar + Sugar
console.log(`Rs.${order.cost()}`); // Rs.70 (50+10+5+5)
```

Decorator vs Inheritance

	Decorator	Inheritance
ఎప్పుడు జోడించు	Runtime	Compile time

Combinations	పొరలుగా ఎన్నైనా	ప్రతి combo కి కొత్త class
Flexibility	ఎక్కువ	తక్కువ

> ***గమనిక:*** *OOP Topic 27 లో* `Symbol.iterator`*, Topic 22 లో wrappers చూశాం - Decorator అదే "wrap చేసి పెంచడం" ఆలోచన.*

---

19. Facade

వివరణ

Facade = ఒక సంక్లిష్టమైన subsystem కి **ఒక సాధారణ ముఖద్వారం (single simple interface)**. లోపలి గజిబిజిని దాచి, client కి ఒక్క సులభమైన method ఇస్తుంది.

Real-life Scenario

> ***Car*** *లో "start" button నొక్కితే - fuel pump, ignition, battery, starter motor అన్నీ లోపల జరుగుతాయి. నీకు ఒక్క button (facade) చాలు, లోపలి 10 steps అక్కర్లేదు.*

Code

```
// సంక్లిష్ట subsystem — చాలా భాగాలు
class FuelPump {
  activate() {
    console.log("Fuel pump ఆన్");
  }
}
class Ignition {
  fire() {
    console.log("Ignition fired");
  }
}
class Starter {
  crank() {
    console.log("Starter motor తిరుగుతోంది");
  }
}

// Facade — అన్నిటినీ దాచి ఒక్క start() ఇస్తుంది
class CarFacade {
  constructor() {
    this.fuel = new FuelPump();
    this.ignition = new Ignition();
    this.starter = new Starter();
  }
  start() {
    console.log("--- Car start అవుతోంది ---");
    this.fuel.activate();
    this.ignition.fire();
    this.starter.crank();
    console.log("🚗 Car ready!");
```

```
  }
}

// Client కి ఒక్క method చాలు — లోపలి complexity అక్కర్లేదు
new CarFacade().start();
// --- Car start అవుతోంది ---
// Fuel pump ఆన్ / Ignition fired / Starter motor తిరుగుతోంది / 🚗 Car ready!
```

Key Points

- Complex library/subsystem ముందు ఒక సులభ layer
- Client ని subsystem మార్పుల నుండి కాపాడుతుంది (loose coupling)
- Adapter interface *మారుస్తుంది*; Facade interface ని *సులభం* చేస్తుంది

---

20. Flyweight

వివరణ

Flyweight = వేలాది objects ఉన్నప్పుడు, common data (intrinsic) ని share చేసి memory ఆదా చేయడం. మారే data (extrinsic) మాత్రం బయట ఉంచుతాం.

Real-life Scenario

> ***అడవి*** *లో 10 లక్షల చెట్లు. ప్రతి చెట్టుకి texture, color image (2MB) విడిగా store చేస్తే memory పేలుతుంది. బదులుగా "Oak" type data ఒక్కసారి store చేసి, అన్ని Oak చెట్లు దాన్ని share చేస్తాయి; ప్రతి చెట్టుకి కేవలం (x, y) position మాత్రం.*

Code

```
// Flyweight — share అయ్యే intrinsic data (type, color, texture)
class TreeType {
  constructor(name, color) {
    this.name = name;
    this.color = color;
  }
  render(x, y) {
    console.log(`${this.color} ${this.name} @(${x},${y})`);
  }
}

// Factory — ఒకే type ని పంచుతుంది (cache)
class TreeFactory {
  static #types = new Map();
  static getType(name, color) {
    const key = `${name}_${color}`;
    if (!TreeFactory.#types.has(key)) {
      TreeFactory.#types.set(key, new TreeType(name, color));
    }
    return TreeFactory.#types.get(key); // ఉన్నదే share
  }
  static get typeCount() {
```

```
    return TreeFactory.#types.size;
  }
}

// చెట్టు – కేవలం extrinsic (position) + shared type reference
class Tree {
  constructor(x, y, type) {
    this.x = x;
    this.y = y;
    this.type = type;
  }
  render() {
    this.type.render(this.x, this.y);
  }
}

const forest = [];
for (let i = 0; i < 1000; i++) {
  const type = TreeFactory.getType("Oak", "Green"); // అన్నీ ఒకే object share
  forest.push(new Tree(i, i * 2, type));
}
forest[0].render(); // Green Oak @(0,0)
forest[999].render(); // Green Oak @(999,1998)
console.log(
  `Trees: ${forest.length}, unique TreeType objects: ${TreeFactory.typeCount}`,
);
// Trees: 1000, unique TreeType objects: 1  (memory ఆదా!)
```

Key Points

- **Intrinsic** (share అయ్యేది) vs **Extrinsic** (ఒక్కో object కి ప్రత్యేకం) విడదీయి
- వేలాది సారూప్య objects ఉన్నప్పుడు మాత్రమే - లేకపోతే over-engineering
- Text editors (character glyphs), games (particles), maps - Flyweight

---

21. Proxy

వివరణ

Proxy = అసలు object కి బదులు నిలిచే **ప్రతినిధి (stand-in)**. Access ని control చేస్తుంది - lazy loading, permission check, caching, logging కోసం.

Real-life Scenario

> ***Credit card*** *= నీ bank account కి proxy. షాప్ లో account నేరుగా ఇవ్వవు; card (proxy) ద్వారా access - అది limit check, security చేస్తుంది.*

Code

```
// అసలు (ఖరీదైన) object
class RealImage {
  constructor(filename) {
```

```
    this.filename = filename;
    this.#loadFromDisk(); // ఖరీదైన operation
  }
  #loadFromDisk() {
    console.log(`💾 ${this.filename} disk నుండి load అవుతోంది...`);
  }
  display() {
    console.log(`🖼️ ${this.filename} చూపిస్తోంది`);
  }
}

// Proxy — అవసరమైనప్పుడే (lazy) అసలు object ని create చేస్తుంది
class ImageProxy {
  constructor(filename) {
    this.filename = filename;
    this.real = null;
  }
  display() {
    if (!this.real) this.real = new RealImage(this.filename); // మొదటిసారే load
    this.real.display();
  }
}

const img = new ImageProxy("photo.jpg"); // ఇంకా load కాలేదు (వేగం)
console.log("Image object తయారైంది, కానీ disk touch కాలేదు.");
img.display(); // ఇప్పుడు load + display (మొదటిసారి)
img.display(); // ఇప్పటికే load అయింది — మళ్ళీ load కాదు (cached)
```

Proxy రకాలు

రకం	పని
Virtual Proxy	Lazy loading (ఖరీదైన object ని ఆలస్యంగా)
Protection Proxy	Access/permission check
Caching Proxy	ఫలితాలని cache
Logging Proxy	Calls ని log

> ***గమనిక:*** *JavaScript లో native `Proxy` object కూడా ఉంది (get/set traps) - ఇదే ఆలోచన language level లో.*

Part 5 — Behavioral Patterns

> *Behavioral patterns = **objects ఒకరితో ఒకరు ఎలా మాట్లాడుకోవాలి, బాధ్యతలు ఎలా పంచుకోవాలి** అనేది చెప్పేవి.*

---

22. Chain of Responsibility

వివరణ

Chain of Responsibility = ఒక request ని handlers **గొలుసు (chain)** గుండా పంపడం. ఒక్కో handler "ఇది నా పని అవునా?" చూసి, కాకపోతే తర్వాతి వాడికి పంపుతుంది.

Real-life Scenario

> ***Customer support**: నీ complaint → Level 1 agent → పరిష్కారం కాకపోతే → Manager → కాకపోతే → Director. ఒక్కో స్థాయి తనవల్ల అయ్యేది చేస్తుంది, లేకపోతే పైకి పంపుతుంది.*

Code

```
class Handler {
  setNext(handler) {
    this.next = handler;
    return handler;
  } // chaining
  handle(amount) {
    if (this.next) return this.next.handle(amount);
    console.log(`Rs.${amount} ఎవరూ approve చేయలేకపోయారు!`);
  }
}

class TeamLead extends Handler {
  handle(amount) {
    if (amount <= 1000)
      return console.log(`TeamLead Rs.${amount} approve చేశారు`);
    return super.handle(amount); // పైకి పంపు
  }
}
class Manager extends Handler {
  handle(amount) {
    if (amount <= 10000)
      return console.log(`Manager Rs.${amount} approve చేశారు`);
    return super.handle(amount);
  }
}
class Director extends Handler {
  handle(amount) {
    if (amount <= 100000)
      return console.log(`Director Rs.${amount} approve చేశారు`);
    return super.handle(amount);
  }
}

// గొలుసు కట్టడం: TeamLead → Manager → Director
const lead = new TeamLead();
lead.setNext(new Manager()).setNext(new Director());

lead.handle(500); // TeamLead Rs.500 approve చేశారు
lead.handle(5000); // Manager Rs.5000 approve చేశారు
lead.handle(50000); // Director Rs.50000 approve చేశారు
lead.handle(500000); // Rs.500000 ఎవరూ approve చేయలేకపోయారు!
```

Key Points

- Sender కి ఏ handler పని చేస్తుందో తెలియదు (loose coupling)
- Handlers ని runtime లో మార్చవచ్చు/జోడించవచ్చు
- Middleware (Express.js), event bubbling, approval flows - ఇదే pattern

---

23. Command

వివరణ

Command = ఒక request ని **object** గా mార్చడం. దీనివల్ల requests ని store, queue, undo, log చేయవచ్చు.

Real-life Scenario

> ***Restaurant order slip**: నువ్వు చెప్పింది waiter ఒక slip (command object) గా రాస్తాడు. ఆ slip ని queue చేయవచ్చు, cancel చేయవచ్చు, kitchen కి పంపవచ్చు. Order = object.*

Code

```
// Receiver - అసలు పని చేసేది
class Light {
  on() {
    console.log("💡 Light ఆన్");
  }
  off() {
    console.log("🌑 Light ఆఫ్");
  }
}

// Commands - ఒక్కో action ఒక object (execute + undo)
class LightOnCommand {
  constructor(light) {
    this.light = light;
  }
  execute() {
    this.light.on();
  }
  undo() {
    this.light.off();
  }
}
class LightOffCommand {
  constructor(light) {
    this.light = light;
  }
  execute() {
    this.light.off();
  }
  undo() {
    this.light.on();
  }
```

```
}

// Invoker - commands ని run చేసి history ఉంచుతుంది (undo కోసం)
class RemoteControl {
  constructor() {
    this.history = [];
  }
  press(command) {
    command.execute();
    this.history.push(command);
  }
  undoLast() {
    const cmd = this.history.pop();
    if (cmd) {
      console.log("↩️ Undo:");
      cmd.undo();
    }
  }
}

const light = new Light();
const remote = new RemoteControl();
remote.press(new LightOnCommand(light)); // 💡 Light ఆన్
remote.press(new LightOffCommand(light)); // 🌑 Light ఆఫ్
remote.undoLast(); // ↩️ Undo: → 💡 Light ఆన్ (last command reverse)
```

Key Points

- Action = object → undo/redo, queue, macro, logging సాధ్యం
- Invoker (remote) కి receiver (light) ఏం చేస్తుందో తెలియదు
- Undo/redo, task queues, transactions - Command pattern

---

24. Iterator

వివరణ

Iterator = ఒక collection లోని elements ని, లోపలి structure చూపించకుండా, ఒక్కొక్కటిగా traverse చేసే మార్గం. (JS లో `Symbol.iterator` - OOP Topic 27.)

Real-life Scenario

> ***TV remote*** *లో channel up button - TV లోపల channels ఎలా store అయ్యాయో నీకు అక్కర్లేదు, "next" నొక్కితే తర్వాతిది వస్తుంది. అదే iterator.*

Code

```
class Playlist {
  #songs = [];
  add(song) {
    this.#songs.push(song);
    return this;
```

```
  }

  // Symbol.iterator — for...of ని enable చేస్తుంది (internal array దాచి)
  [Symbol.iterator]() {
    let i = 0;
    const songs = this.#songs;
    return {
      next() {
        return i < songs.length
          ? { value: songs[i++], done: false }
          : { value: undefined, done: true };
      },
    };
  }
}

const playlist = new Playlist();
playlist.add("పాట 1").add("పాట 2").add("పాట 3");

// లోపల array అని client కి తెలియదు — కేవలం iterate చేస్తాడు
for (const song of playlist) console.log("▶️ " + song);
// ▶️ పాట 1 / ▶️ పాట 2 / ▶️ పాట 3

console.log([...playlist].length); // 3 (spread కూడా పనిచేస్తుంది)
```

Key Points

- Collection యొక్క internal structure (array? tree? linked list?) దాచుతుంది
- ఒకే interface (`next`) తో ఏ collection అయినా traverse
- JS లో `Symbol.iterator` + generators (`function*`) native support

---

25. Mediator

వివరణ

Mediator = objects నేరుగా ఒకరితో ఒకరు మాట్లాడకుండా, ఒక **మధ్యవర్తి (mediator)** ద్వారా communicate చేయడం. Many-to-many సంబంధాలని simplify చేస్తుంది.

Real-life Scenario

> ***Air Traffic Control (ATC)***: *విమానాలు ఒకదానితో ఒకటి నేరుగా మాట్లాడవు - అన్నీ ATC tower (mediator) తో మాట్లాడతాయి. లేకపోతే గందరగోళం, ప్రమాదం.*

Code

```
// Mediator — అందరి మధ్య సందేశాలు పంపుతుంది
class ChatRoom {
  constructor() {
    this.users = [];
  }
  register(user) {
```

```
    user.room = this;
    this.users.push(user);
  }
  send(message, from) {
    this.users
      .filter((u) => u !== from) // పంపినవాడికి తప్ప అందరికీ
      .forEach((u) => u.receive(message, from.name));
  }
}

// Colleagues — నేరుగా కాకుండా room ద్వారా మాట్లాడతారు
class User {
  constructor(name) {
    this.name = name;
    this.room = null;
  }
  send(message) {
    console.log(`${this.name} పంపారు: ${message}`);
    this.room.send(message, this);
  }
  receive(message, fromName) {
    console.log(`  ${this.name} కి [${fromName}]: ${message}`);
  }
}

const room = new ChatRoom();
const a = new User("Yaswanth"),
  b = new User("Chaitanya"),
  c = new User("Priya");
[a, b, c].forEach((u) => room.register(u));

a.send("అందరికీ నమస్తే!");
// Yaswanth పంపారు: అందరికీ నమస్తే!
//   Chaitanya కి [Yaswanth]: అందరికీ నమస్తే!
//   Priya కి [Yaswanth]: అందరికీ నమస్తే!
```

Key Points

- N objects మధ్య N×N connections → N objects + 1 mediator
- Objects ఒకదాని గురించి ఒకటి తెలియక్కర్లేదు (mediator కి మాత్రమే తెలుసు)
- Chat rooms, UI dialogs (fields మధ్య), event buses

---

26. Memento

వివరణ

Memento = ఒక object యొక్క state ని (encapsulation పాడవకుండా) బయట save చేసి, తర్వాత restore (undo) చేయడం.

Real-life Scenario

> ***Game save point**: boss fight ముందు save చేస్తావు. చనిపోతే ఆ save point కి తిరిగి వస్తావు. State ని snapshot గా దాచడం.*

Code

```
// Originator — ఎవరి state save/restore అవుతుందో
class TextEditor {
  #content = "";
  type(text) {
    this.#content += text;
  }
  getContent() {
    return this.#content;
  }

  save() {
    return { snapshot: this.#content };
  } // Memento (state copy)
  restore(memento) {
    this.#content = memento.snapshot;
  }
}

// Caretaker — mementos ని ఉంచుతుంది (state లోపల చూడకుండా)
class History {
  #stack = [];
  push(memento) {
    this.#stack.push(memento);
  }
  pop() {
    return this.#stack.pop();
  }
}

const editor = new TextEditor();
const history = new History();

editor.type("నమస్తే ");
history.push(editor.save()); // checkpoint 1
editor.type("ప్రపంచం");
console.log(editor.getContent()); // నమస్తే ప్రపంచం

editor.restore(history.pop()); // undo — checkpoint 1 కి
console.log(editor.getContent()); // నమస్తే
```

Key Points

- State ని snapshot గా బయట store చేస్తాం (encapsulation break అవ్వదు)
- Caretaker snapshot లోపలికి చూడదు - కేవలం ఉంచి, తిప్పి ఇస్తుంది
- Undo/redo, checkpoints, transactions rollback

---

27. Observer

వివరణ

Observer = ఒక object (subject) state మారినప్పుడు, దానిపై ఆధారపడిన అందరికీ (observers) **ఆటోమేటిక్‌గా notify** చేయడం. Publish-Subscribe.

Real-life Scenario

> ***YouTube channel***: *నువ్వు subscribe చేస్తే, కొత్త video వచ్చినప్పుడు అందరు subscribers కి notification. Channel ప్రతి subscriber ని విడిగా పిలవదు - అందరికీ broadcast.*

Code

```
// Subject – observers ని ఉంచి, మారినప్పుడు notify చేస్తుంది
class Channel {
  constructor(name) {
    this.name = name;
    this.subscribers = [];
  }
  subscribe(fn) {
    this.subscribers.push(fn);
  }
  unsubscribe(fn) {
    this.subscribers = this.subscribers.filter((s) => s !== fn);
  }
  upload(video) {
    console.log(`📢 ${this.name}: "${video}" upload అయింది`);
    this.subscribers.forEach((fn) => fn(video)); // అందరికీ notify
  }
}

const channel = new Channel("Telugu Coding");

const yaswanth = (video) => console.log(`  Yaswanth చూస్తున్నాడు: ${video}`);
const priya = (video) => console.log(`  Priya చూస్తోంది: ${video}`);

channel.subscribe(yaswanth);
channel.subscribe(priya);
channel.upload("LLD in Telugu");
// 📢 Telugu Coding: "LLD in Telugu" upload అయింది
//   Yaswanth చూస్తున్నాడు: LLD in Telugu
//   Priya చూస్తోంది: LLD in Telugu

channel.unsubscribe(priya);
channel.upload("Design Patterns"); // ఇప్పుడు Yaswanth కి మాత్రమే
```

Key Points

- Subject, observers loosely coupled - subject కి observers ఎవరో వివరంగా తెలియదు
- Event systems, MVC, reactive UI (React state), RxJS - అంతా Observer

- Memory leak జాగ్రత్త: అవసరం లేని observers ని unsubscribe చేయి

---

28. State

వివరణ

State = object యొక్క behaviour దాని **internal state** బట్టి మారడం - అదీ పెద్ద `if/else` లేకుండా. ప్రతి state ఒక class.

Real-life Scenario

> ***Traffic signal**: Red → Green → Yellow → Red. ఒక్కో state లో "next()" వేరే విధంగా పనిచేస్తుంది. Signal ఒకటే object, కానీ state బట్టి behaviour మారుతుంది.*

Code

```
// ఒక్కో state ఒక object - next() ఏ state కి వెళ్ళాలో అదే చెప్తుంది
const RedState = {
  name: "🔴 Red (ఆగు)",
  next: () => GreenState,
};
const GreenState = {
  name: "🟢 Green (వెళ్ళు)",
  next: () => YellowState,
};
const YellowState = {
  name: "🟡 Yellow (నెమ్మది)",
  next: () => RedState,
};

class TrafficSignal {
  constructor() {
    this.state = RedState;
  }
  change() {
    this.state = this.state.next();
  } // state తనే తర్వాతిది నిర్ణయిస్తుంది
  show() {
    console.log(this.state.name);
  }
}

const signal = new TrafficSignal();
signal.show(); // 🔴 Red (ఆగు)
signal.change();
signal.show(); // 🟢 Green (వెళ్ళు)
signal.change();
signal.show(); // 🟡 Yellow (నెమ్మది)
signal.change();
signal.show(); // 🔴 Red (ఆగు) - cycle
```

State vs Strategy

	State	Strategy
ఎవరు మారుస్తారు	Object తనే state మారుస్తుంది	Client strategy ఎంచుకుంటాడు
States ఒకదానికొకటి తెలుసా	అవును (next state)	లేదు (స్వతంత్రం)
ఉదా	Traffic signal, order status	Payment method, sort algorithm

---

29. Strategy

వివరణ

Strategy = ఒకే పనికి **అనేక algorithms** ని విడి objects గా చేసి, runtime లో మార్చుకోవడం. `if/else` బదులు polymorphism.

Real-life Scenario

> ***Google Maps** లో ఒకే గమ్యానికి: కారు route, నడక route, bus route. ఒకే "navigate" - కానీ నువ్వు ఎంచుకున్న **strategy** బట్టి లెక్క మారుతుంది.*

Code

```
// Strategies — ఒకే contract (calculate), వేరే logic
const carStrategy = (km) => ({ time: km / 60, mode: "🚗 కారు" });
const walkStrategy = (km) => ({ time: km / 5, mode: "🚶 నడక" });
const bikeStrategy = (km) => ({ time: km / 40, mode: "🏍 బైక్" });

// Context — ఏ strategy అనేది బయట నుండి inject/switch
class RoutePlanner {
  setStrategy(strategy) {
    this.strategy = strategy;
    return this;
  }
  plan(km) {
    const { time, mode } = this.strategy(km);
    console.log(`${mode}: ${km}km → ${time.toFixed(1)} గంటలు`);
  }
}

const planner = new RoutePlanner();
planner.setStrategy(carStrategy).plan(120); // 🚗 కారు: 120km → 2.0 గంటలు
planner.setStrategy(walkStrategy).plan(120); // 🚶 నడక: 120km → 24.0 గంటలు
planner.setStrategy(bikeStrategy).plan(120); // 🏍 బైక్: 120km → 3.0 గంటలు
```

Key Points

- పెరుగుతున్న `if/else` / `switch` = Strategy అడుగుతోంది (OCP)
- Algorithm ని runtime లో మార్చవచ్చు
- Payment methods, sorting, compression, pricing - Strategy classics

- Topic 9 (Program to Interface) యొక్క నేరు అమలు ఇదే

---

30. Template Method

వివరణ

Template Method = ఒక algorithm యొక్క **అస్థిపంజరం (skeleton)** ని parent లో fix చేసి, కొన్ని steps ని children override చేసేలా వదలడం. క్రమం మారదు, steps మారతాయి.

Real-life Scenario

> ***Tea vs Coffee** తయారీ: నీళ్ళు మరిగించు → [ఏదో కలుపు] → కప్‌లో పోయు → [ఏదో జోడించు]. Skeleton ఒకటే; కలిపే పదార్థం మాత్రం tea/coffee బట్టి మారుతుంది.*

Code

```
// Parent — algorithm skeleton fix చేస్తుంది (template method)
class Beverage {
  prepare() {
    // ఈ క్రమం (template) children మార్చలేరు
    this.boilWater();
    this.brew(); // step — child నిర్ణయిస్తుంది
    this.pourInCup();
    this.addCondiments(); // step — child నిర్ణయిస్తుంది
  }
  boilWater() {
    console.log("💧 నీళ్ళు మరిగిస్తోంది");
  }
  pourInCup() {
    console.log("☕ కప్‌లో పోస్తోంది");
  }
  brew() {
    throw new Error("brew() override చేయాలి");
  }
  addCondiments() {
    throw new Error("addCondiments() override చేయాలి");
  }
}

class Tea extends Beverage {
  brew() {
    console.log("🍵 టీ పొడి వేస్తోంది");
  }
  addCondiments() {
    console.log("🍋 నిమ్మ జోడిస్తోంది");
  }
}
class Coffee extends Beverage {
  brew() {
    console.log("☕ కాఫీ పొడి వేస్తోంది");
  }
```

```
  addCondiments() {
    console.log("🥛 పాలు + చక్కెర జోడిస్తోంది");
  }
}

console.log("--- Tea ---");
new Tea().prepare();
console.log("--- Coffee ---");
new Coffee().prepare();
// క్రమం ఒకటే, కానీ brew + condiments వేరు
```

Key Points

- Algorithm యొక్క క్రమం fix; కొన్ని steps మాత్రం subclass నిర్ణయిస్తుంది
- Code duplication తగ్గుతుంది (common steps parent లో)
- "Don't call us, we'll call you" (Hollywood principle) - parent children ని పిలుస్తుంది

---

31. Visitor

వివరణ

Visitor = objects structure ని మార్చకుండా, వాటిపై **కొత్త operations** జోడించడం. Operation ని ఒక "visitor" object లో పెడతాం.

Real-life Scenario

> ***Tax auditor** ఇంటింటికీ వెళ్తాడు (visit). ప్రతి ఇల్లు (shop, house, factory) తనని తాను మార్చుకోదు - auditor తన లెక్క (operation) తెచ్చి ప్రతిదానిపై apply చేస్తాడు.*

Code

```
// Elements — accept(visitor) మాత్రమే ఉంటుంది (operations బయట)
class Circle {
  constructor(r) {
    this.r = r;
  }
  accept(visitor) {
    return visitor.visitCircle(this);
  }
}
class Rectangle {
  constructor(w, h) {
    this.w = w;
    this.h = h;
  }
  accept(visitor) {
    return visitor.visitRectangle(this);
  }
}

// Visitor — కొత్త operation (area). Shapes మార్చకుండా జోడించాం.
```

```
const areaVisitor = {
  visitCircle: (c) => Math.PI * c.r ** 2,
  visitRectangle: (r) => r.w * r.h,
};
// మరో కొత్త operation - మళ్ళీ shapes touch చేయలేదు
const perimeterVisitor = {
  visitCircle: (c) => 2 * Math.PI * c.r,
  visitRectangle: (r) => 2 * (r.w + r.h),
};

const shapes = [new Circle(5), new Rectangle(4, 6)];
shapes.forEach((s) => {
  console.log(
    `Area: ${s.accept(areaVisitor).toFixed(2)}, Perimeter:
${s.accept(perimeterVisitor).toFixed(2)}`,
  );
});
// Area: 78.54, Perimeter: 31.42
// Area: 24.00, Perimeter: 20.00
```

Key Points

- కొత్త operation జోడించాలంటే కొత్త visitor - element classes touch చేయక్కర్లేదు (OCP)
- కానీ కొత్త element type జోడిస్తే అన్ని visitors మార్చాలి (trade-off)
- Compilers (AST traversal), document exporters - Visitor

---

32. Interpreter

వివరణ

Interpreter = ఒక చిన్న భాష (grammar) కి, దాని వాక్యాలను **evaluate** చేసే classes రాయడం. ప్రతి grammar rule ఒక class; అన్నిటికీ `interpret()` ఉంటుంది.

Real-life Scenario

> ***Calculator*** *లో "5 + 3 - 2" అని type చేస్తే, అది ఆ expression ని అర్థం చేసుకొని (interpret) 6 అని లెక్కిస్తుంది. లేదా Google Translate ఒక వాక్యాన్ని అర్థం చేసుకున్నట్లు - grammar ప్రకారం interpret చేయడం.*

Code

```
// Terminal expression - number (ఆకు / leaf)
class NumberExpr {
  constructor(value) {
    this.value = value;
  }
  interpret() {
    return this.value;
  }
}

// Non-terminal expressions - operations (కొమ్మలు)
```

```javascript
class AddExpr {
  constructor(left, right) {
    this.left = left;
    this.right = right;
  }
  interpret() {
    return this.left.interpret() + this.right.interpret();
  }
}
class SubtractExpr {
  constructor(left, right) {
    this.left = left;
    this.right = right;
  }
  interpret() {
    return this.left.interpret() - this.right.interpret();
  }
}

// "5 + 3 - 2" ని expression tree గా చేసి interpret: (5 + 3) - 2
const expr = new SubtractExpr(
  new AddExpr(new NumberExpr(5), new NumberExpr(3)),
  new NumberExpr(2),
);
console.log(expr.interpret()); // 6

// చిన్న parser - RPN (postfix) input ని tree గా build చేసి interpret
function interpretRPN(tokens) {
  const stack = [];
  for (const token of tokens) {
    if (token === "+") {
      const b = stack.pop(),
        a = stack.pop();
      stack.push(new AddExpr(a, b));
    } else if (token === "-") {
      const b = stack.pop(),
        a = stack.pop();
      stack.push(new SubtractExpr(a, b));
    } else stack.push(new NumberExpr(Number(token)));
  }
  return stack.pop().interpret();
}

console.log(interpretRPN("5 3 + 2 -".split(" "))); // 6  ((5+3)-2)
console.log(interpretRPN("10 20 +".split(" "))); // 30
```

Key Points

- ప్రతి grammar rule = ఒక class; expression = ఆ classes తో కట్టిన tree
- SQL parsers, regular expressions, calculators, rule engines - Interpreter
- ⚠️ Grammar పెద్దదైతే classes పేలతాయి - complex భాషలకి parser generator మంచిది

Part 6 — Real-world Case Studies

> *ఇప్పుడు నేర్చుకున్న principles + patterns ని కలిపి, నిజమైన interview problems solve చేద్దాం. ప్రతి case study లో: requirements → entities → patterns → tested code.*

---

33. Parking Lot System

Requirements

- చాలా floors, ఒక్కో floor కి slots
- Vehicle types: Bike, Car (వేర్వేరు slot sizes)
- Park చేస్తే ticket, exit అయితే fee (గంటల ప్రకారం)

Entities + Patterns

Entity	పని	Pattern
`VehicleFactory`	vehicle create	Factory
`FeeStrategy`	fee లెక్క	Strategy
`ParkingLot`	slots manage	(Singleton-గా వాడవచ్చు)

Code

```
// --- Vehicle types (Factory pattern) ---
class Vehicle {
  constructor(number, type, spotSize) {
    this.number = number;
    this.type = type;
    this.spotSize = spotSize;
  }
}
class VehicleFactory {
  static create(number, type) {
    if (type === "bike") return new Vehicle(number, "bike", 1);
    if (type === "car") return new Vehicle(number, "car", 2);
    throw new Error("Unknown vehicle type");
  }
}

// --- Fee strategy (Strategy pattern) ---
const flatFee = (hours) => hours * 20;
const rampFee = (hours) => (hours <= 2 ? hours * 30 : 60 + (hours - 2) * 15);

// --- Ticket ---
class Ticket {
  constructor(vehicle, spotId, entryHour) {
    this.vehicle = vehicle;
    this.spotId = spotId;
    this.entryHour = entryHour;
```

```
  }
}

// --- Parking Lot (core) ---
class ParkingLot {
  constructor(totalSpots, feeStrategy = flatFee) {
    this.spots = Array.from({ length: totalSpots }, (_, i) => ({
      id: i,
      free: true,
    }));
    this.feeStrategy = feeStrategy;
  }
  park(vehicle, entryHour) {
    const spot = this.spots.find((s) => s.free);
    if (!spot) {
      console.log("❌ Parking full!");
      return null;
    }
    spot.free = false;
    console.log(`🅿️ ${vehicle.type} ${vehicle.number} → spot ${spot.id}`);
    return new Ticket(vehicle, spot.id, entryHour);
  }
  unpark(ticket, exitHour) {
    const spot = this.spots[ticket.spotId];
    spot.free = true;
    const hours = exitHour - ticket.entryHour;
    const fee = this.feeStrategy(hours);
    console.log(`🚗 ${ticket.vehicle.number} exit. ${hours}h → Rs.${fee}`);
    return fee;
  }
  get availableCount() {
    return this.spots.filter((s) => s.free).length;
  }
}

// --- Simulation ---
const lot = new ParkingLot(2, rampFee);
const t1 = lot.park(VehicleFactory.create("AP01AB1234", "car"), 10);
const t2 = lot.park(VehicleFactory.create("AP02XY9999", "bike"), 11);
lot.park(VehicleFactory.create("AP03ZZ0000", "car"), 12); // ❌ Parking full!
console.log(`Available: ${lot.availableCount}`); // 0
lot.unpark(t1, 15); // 5h → Rs.105 (60 + 3*15)
console.log(`Available: ${lot.availableCount}`); // 1
```

గమనిక

- కొత్త vehicle type → `VehicleFactory` లో మాత్రమే మార్పు (OCP)
- కొత్త pricing → కొత్త fee strategy, `ParkingLot` touch వద్దు
- Slot allocation ని Strategy గా చేస్తే "nearest spot" లాంటి logic జోడించవచ్చు

---

34. LRU Cache

వివరణ

LRU (Least Recently Used) Cache = fixed size cache; నిండినప్పుడు, **చాలా కాలం వాడని** item ని తీసేస్తుంది. Interview favourite.

ముఖ్య ఆలోచన

> `Map` *(JavaScript) insertion order ని గుర్తుంచుకుంటుంది. Access అయిన item ని delete చేసి మళ్ళీ add చేస్తే అది "అత్యంత recent" అవుతుంది. మొదటిది = least recent.*

Code

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.map = new Map(); // insertion order = usage order
  }

  get(key) {
    if (!this.map.has(key)) return -1;
    const value = this.map.get(key);
    this.map.delete(key); // తీసి...
    this.map.set(key, value); // ...చివర పెట్టు (most recent)
    return value;
  }

  put(key, value) {
    if (this.map.has(key)) this.map.delete(key);
    else if (this.map.size >= this.capacity) {
      const lru = this.map.keys().next().value; // మొదటిది = least recent
      this.map.delete(lru);
      console.log(`🗑️ Evicted key: ${lru}`);
    }
    this.map.set(key, value);
  }
}

const cache = new LRUCache(2);
cache.put("a", 1);
cache.put("b", 2);
console.log(cache.get("a")); // 1 (ఇప్పుడు "a" recent, "b" old)
cache.put("c", 3); // 🗑️ Evicted key: b (b least recently used)
console.log(cache.get("b")); // -1 (పోయింది)
console.log(cache.get("c")); // 3
```

Key Points

- `Map` తో O(1) get/put (order preserved) - production లో doubly linked list + hash map
- Access చేసినప్పుడు item ని "refresh" చేయడం (delete + re-add) కీలకం

- Redis, browser cache, DB query cache - LRU eviction

---

35. Vending Machine (State pattern)

వివరణ

Vending machine states: **NoMoney → HasMoney → Dispensing**. ఒక్కో state లో buttons వేరుగా behave చేస్తాయి. State pattern (Topic 28) కి perfect fit.

Code

```
class VendingMachine {
  constructor(stock) {
    this.stock = stock;
    this.balance = 0;
    this.state = "NO_MONEY";
  }

  insertMoney(amount) {
    if (this.state === "NO_MONEY") {
      this.balance += amount;
      this.state = "HAS_MONEY";
      console.log(`💰 Rs.${amount} inserted. Balance: Rs.${this.balance}`);
    } else {
      this.balance += amount;
      console.log(`💰 మరో Rs.${amount}. Balance: Rs.${this.balance}`);
    }
  }

  selectItem(price) {
    if (this.state !== "HAS_MONEY") {
      console.log("❌ ముందు డబ్బు వేయండి!");
      return;
    }
    if (this.stock <= 0) {
      console.log("❌ Stock లేదు!");
      return;
    }
    if (this.balance < price) {
      console.log(`❌ Rs.${price - this.balance} తక్కువ`);
      return;
    }

    this.state = "DISPENSING";
    this.balance -= price;
    this.stock--;
    console.log(
      `🥤 Item dispensed! మిగిలింది: Rs.${this.balance}, Stock: ${this.stock}`,
    );
    this.state = this.balance > 0 ? "HAS_MONEY" : "NO_MONEY";
  }
```

```
}

const machine = new VendingMachine(1);
machine.selectItem(40); // ❌ ముందు డబ్బు వేయండి!
machine.insertMoney(50); // 💰 Rs.50 inserted. Balance: Rs.50
machine.selectItem(40); // 🥤 Item dispensed! మిగిలింది: Rs.10, Stock: 0
machine.insertMoney(50); // 💰 Rs.50. Balance: Rs.60
machine.selectItem(40); // ❌ Stock లేదు!
```

Key Points

- State machine = valid transitions ని స్పష్టంగా చూపిస్తుంది
- పెద్ద systems లో ఒక్కో state ని విడి class చేస్తే (Topic 28) if/else తగ్గుతుంది
- ATM, order lifecycle, game character states - state machines

---

36. Rate Limiter

వివరణ

Rate Limiter = ఒక user నిర్ణీత సమయంలో ఎన్ని requests చేయవచ్చో పరిమితి. ఇక్కడ **Token Bucket** algorithm - ప్రతి secondకి tokens నిండుతాయి, ప్రతి request ఒక token తింటుంది.

Code

```
class TokenBucket {
  constructor(capacity, refillPerSec) {
    this.capacity = capacity;
    this.tokens = capacity;
    this.refillPerSec = refillPerSec;
  }

  // elapsedSec = గత call నుండి గడిచిన సమయం (test కోసం parameter గా ఇస్తున్నాం)
  allow(elapsedSec = 0) {
    // సమయం గడిచిన కొద్దీ tokens నింపు (capacity దాటకుండా)
    this.tokens = Math.min(
      this.capacity,
      this.tokens + elapsedSec * this.refillPerSec,
    );
    if (this.tokens >= 1) {
      this.tokens -= 1;
      return true; // request allow
    }
    return false; // rate limited
  }
}

// 3 tokens capacity, secondకి 1 token refill
const limiter = new TokenBucket(3, 1);
console.log(limiter.allow()); // true  (3→2)
console.log(limiter.allow()); // true  (2→1)
console.log(limiter.allow()); // true  (1→0)
```

```
console.log(limiter.allow()); // false (0 tokens - rate limited!)
console.log(limiter.allow(2)); // true  (2 sec గడిచింది → 2 tokens refill → allow)
```

Rate Limiting algorithms

Algorithm	ఆలోచన
Token Bucket	tokens నిండుతాయి, request token తింటుంది (bursts ok)
Leaky Bucket	requests ఒక rate లో "leak" అవుతాయి
Fixed Window	window కి X requests
Sliding Window	rolling window లో count

Key Points

- Token Bucket bursts ని allow చేస్తుంది (tokens పోగుపడితే)
- API gateways, login attempts, DDoS protection - rate limiting
- Production: time ని `Date.now()` తో; ఇక్కడ testable గా `elapsedSec` inject చేశాం

---

37. Notification Service (Observer + Strategy)

వివరణ

రెండు patterns కలిపి: **Observer** (event వచ్చినప్పుడు subscribers కి notify) + **Strategy** (ఏ channel - SMS/Email/Push - అనేది swappable).

Code

```
// --- Strategy: channels (ఒకే send contract) ---
const smsChannel = (user, msg) => console.log(`📱 SMS → ${user}: ${msg}`);
const emailChannel = (user, msg) => console.log(`📧 Email → ${user}: ${msg}`);
const pushChannel = (user, msg) => console.log(`🔔 Push → ${user}: ${msg}`);

// --- Observer: subscribers ఒక్కో channel strategy తో register ---
class NotificationService {
  constructor() {
    this.subscribers = [];
  }
  subscribe(user, channel) {
    this.subscribers.push({ user, channel });
  }
  // Event వచ్చినప్పుడు అందరికీ వాళ్ళ channel లో notify
  notifyAll(message) {
    this.subscribers.forEach(({ user, channel }) => channel(user, message));
  }
}

const service = new NotificationService();
service.subscribe("Yaswanth", smsChannel);
service.subscribe("Chaitanya", emailChannel);
```

```
service.subscribe("Priya", pushChannel);

service.notifyAll("మీ order ship అయింది! 📦");
// 📱 SMS → Yaswanth: మీ order ship అయింది! 📦
// 📧 Email → Chaitanya: మీ order ship అయింది! 📦
// 🔔 Push → Priya: మీ order ship అయింది! 📦
```

గమనిక

- **Observer** = "ఎవరికి పంపాలి" (subscribers)
- **Strategy** = "ఎలా పంపాలి" (channel)
- కొత్త channel (WhatsApp) = కొత్త strategy function, service touch వద్దు
- Real-world లో patterns ఒంటరిగా కాదు, కలిసి పనిచేస్తాయి

---

38. Elevator System

Requirements

- Multiple floors, floor requests
- ఒక్కో request ని దగ్గరి క్రమంలో serve చేయాలి
- Direction (UP/DOWN/IDLE) track చేయాలి

Entities

Entity	పని
`Elevator`	floor, direction, requests manage
`Request`	ఏ floor కి కావాలి
(Scheduler)	ఏ request ముందు - strategy గా విడదీయవచ్చు

Code

```
class Elevator {
  constructor() {
    this.floor = 0;
    this.direction = "IDLE";
    this.requests = [];
  }

  request(floor) {
    if (!this.requests.includes(floor)) this.requests.push(floor);
    console.log(`🛗 Floor ${floor} కి request`);
  }

  step() {
    if (!this.requests.length) {
      this.direction = "IDLE";
      return;
    }
    // దగ్గరి floor ని target చేయడం (SCAN algorithm కి simplified)
```

```
    const target = this.requests.sort(
      (a, b) => Math.abs(a - this.floor) - Math.abs(b - this.floor),
    )[0];
    this.direction = target > this.floor ? "UP" : "DOWN";
    this.floor = target;
    this.requests = this.requests.filter((f) => f !== target);
    console.log(`   → Floor ${this.floor} కి చేరింది (${this.direction})`);
  }

  run() {
    while (this.requests.length) this.step();
    this.direction = "IDLE";
    console.log("   IDLE ✅");
  }
}

const lift = new Elevator();
lift.request(3);
lift.request(1);
lift.request(5);
lift.run();
// → Floor 1 కి చేరింది (UP)   [0 నుండి దగ్గరిది]
// → Floor 3 కి చేరింది (UP)
// → Floor 5 కి చేరింది (UP)
// IDLE ✅
```

గమనిక

- Scheduling logic ని Strategy గా విడదీస్తే - SCAN, LOOK, nearest-first algorithms swap చేయవచ్చు
- Real system: multiple elevators + central dispatcher (Mediator)
- State pattern (Topic 28): Moving / Idle / DoorOpen states

---

39. Tic-Tac-Toe

Requirements

- 3×3 board, ఇద్దరు players (X, O)
- Move validate (cell ఖాళీయేనా), win/draw check
- Turn switching

Entities

Entity	పని
`Board`	cells, place(), win/draw check
`Game`	turn manage, move orchestrate
(Player)	symbol (X/O)

Code

```
class Board {
  constructor() {
    this.cells = Array(9).fill(null);
  }

  place(pos, symbol) {
    if (this.cells[pos]) return false; // ఖాళీ కాదు
    this.cells[pos] = symbol;
    return true;
  }

  winner() {
    const lines = [
      [0, 1, 2],
      [3, 4, 5],
      [6, 7, 8], // rows
      [0, 3, 6],
      [1, 4, 7],
      [2, 5, 8], // columns
      [0, 4, 8],
      [2, 4, 6], // diagonals
    ];
    for (const [a, b, c] of lines) {
      if (
        this.cells[a] &&
        this.cells[a] === this.cells[b] &&
        this.cells[a] === this.cells[c]
      )
        return this.cells[a];
    }
    return null;
  }

  isFull() {
    return this.cells.every((c) => c !== null);
  }
}

class Game {
  constructor() {
    this.board = new Board();
    this.turn = "X";
  }

  move(pos) {
    if (!this.board.place(pos, this.turn)) {
      console.log("❌ Cell నిండింది");
      return;
    }
    const w = this.board.winner();
    if (w) {
```

```
      console.log(`🏆 ${w} గెలిచింది!`);
      return;
    }
    if (this.board.isFull()) {
      console.log("🤝 Draw!");
      return;
    }
    this.turn = this.turn === "X" ? "O" : "X"; // turn switch
  }
}

const game = new Game();
// X: 0,1,2 (top row) → X గెలుస్తుంది
[0, 3, 1, 4, 2].forEach((pos) => game.move(pos));
// 🏆 X గెలిచింది!
```

గమనిక

- `Board` (data + rules) ని `Game` (flow) నుండి విడదీయడం = SRP
- Win-check ని Strategy గా చేస్తే N×N board లకి scale అవుతుంది
- AI opponent = Strategy (random / minimax) గా plug చేయవచ్చు

Part 7 — Beyond GoF (Practical LLD Patterns)

> *GoF 23 patterns కాకుండా, నిజ projects లో తరచూ కనిపించే మరికొన్ని ముఖ్య patterns + concepts. వీటిని interviews లో అడగడం common.*

---

40. Dependency Injection (DI)

వివరణ

Dependency Injection = ఒక class తనకి కావలసిన dependencies ని తనే `new` తో create చేయకుండా, **బయట నుండి inject** చేయించుకోవడం. ఇది DIP (SOLID లో "D") యొక్క ఆచరణ - testable, loosely-coupled code కి కీలకం.

Real-life Scenario

> ***కారుకి engine కావాలి - కానీ కారు factory లో engine తయారు చేయదు.*** *Engine బయట తయారై, కారులో అమర్చబడుతుంది (inject). రేపు వేరే engine కావాలంటే కారు design మార్చక్కర్లేదు.*

3 రకాల Injection

```
class Engine {
  start() {
    return "🔧 Engine started";
  }
}

// 1. Constructor Injection (అత్యంత common) — constructor లో ఇవ్వడం
class Car {
```

```
  constructor(engine) {
    this.engine = engine;
  }
  drive() {
    console.log(this.engine.start());
  }
}
new Car(new Engine()).drive(); // 🔧 Engine started

// 2. Setter Injection — object తయారయ్యాక set చేయడం
class Service {
  setLogger(logger) {
    this.logger = logger;
    return this;
  }
  run() {
    this.logger?.log("running...");
  }
}
const consoleLogger = { log: (m) => console.log(`LOG: ${m}`) };
new Service().setLogger(consoleLogger).run(); // LOG: running...
```

Simple DI Container

```
class Engine {
  start() {
    return "🔧 Engine started";
  }
}
class Car {
  constructor(engine) {
    this.engine = engine;
  }
  drive() {
    console.log(this.engine.start());
  }
}

// Container — dependencies register చేసి, auto-wire చేసి resolve చేస్తుంది
class Container {
  #registry = new Map();
  register(name, factory) {
    this.#registry.set(name, factory);
    return this;
  }
  resolve(name) {
    return this.#registry.get(name)(this);
  }
}

const container = new Container();
```

```
container.register("engine", () => new Engine());
container.register("car", (c) => new Car(c.resolve("engine"))); // auto-wire!
container.resolve("car").drive(); // 🔧 Engine started
```

Key Points

- `new` ని class లోపల కాకుండా బయట → mock inject చేసి easy testing
- Constructor injection = mandatory deps; Setter = optional deps
- Angular, NestJS, Spring - DI containers మీద నడుస్తాయి

---

41. Object Pool

వివరణ

Object Pool = ఖరీదైన objects (DB connections, threads) ని ప్రతిసారి create/destroy చేయకుండా, ఒక **pool** లో ఉంచి reuse చేయడం. (Prototype cloning కి బంధువు - creation cost తగ్గించడం.)

Real-life Scenario

> ***Cricket నెట్స్*** *లో balls ఒక bucket లో ఉంటాయి. ప్రతి ball కి కొత్తది కొనరు - bucket నుండి తీసి, వాడాక తిరిగి పెడతారు. అదే object pool.*

Code

```
class DBConnection {
  constructor(id) {
    this.id = id;
  }
  query(sql) {
    console.log(`Conn#${this.id}: ${sql}`);
  }
}

class ConnectionPool {
  #free = [];
  #used = new Set();
  #nextId = 1;

  constructor(size) {
    for (let i = 0; i < size; i++)
      this.#free.push(new DBConnection(this.#nextId++));
  }

  acquire() {
    if (!this.#free.length) throw new Error("Pool exhausted!");
    const conn = this.#free.pop();
    this.#used.add(conn);
    return conn;
  }

  release(conn) {
```

```
    this.#used.delete(conn);
    this.#free.push(conn);
  } // తిరిగి pool కి
  get stats() {
    return `free:${this.#free.length} used:${this.#used.size}`;
  }
}

const pool = new ConnectionPool(2);
const c1 = pool.acquire();
const c2 = pool.acquire();
console.log(pool.stats); // free:0 used:2
c1.query("SELECT * FROM users"); // Conn#2: SELECT * FROM users
pool.release(c1); // c1 ని తిరిగి pool కి
console.log(pool.stats); // free:1 used:1
const c3 = pool.acquire(); // కొత్తది create కాదు — విడుదల అయిన c1 నే reuse
console.log(c3 === c1); // true (same object reused!)
```

Key Points

- Object creation ఖరీదైనప్పుడు (network, threads) reuse తో performance పెరుగుతుంది
- Pool exhausted అయితే: wait, లేదా error, లేదా grow - policy నిర్ణయించాలి
- DB connection pools, thread pools, game object pools

---

42. Null Object

వివరణ

Null Object = `null` return చేసే బదులు, "ఏమీ చేయని" (do-nothing) object return చేయడం. దీనివల్ల client code లో `if (x === null)` checks అవసరం లేదు.

Real-life Scenario

> *Login కాని user కి "Guest" account ఇస్తాం. `null` కాదు - Guest object. దానికి `getName()` = "Guest", permissions = ఏమీ లేవు. Code ఎక్కడా null check చేయక్కర్లేదు.*

Code

```
class RealUser {
  constructor(name) {
    this.name = name;
  }
  getName() {
    return this.name;
  }
  isGuest() {
    return false;
  }
}

// Null Object — null కి బదులు "do-nothing / default" object
```

```
class GuestUser {
  getName() {
    return "Guest";
  }
  isGuest() {
    return true;
  }
}

const db = { 1: new RealUser("Yaswanth") };

function findUser(id) {
  return db[id] || new GuestUser(); // null return చేయం!
}

// Client కి null check అక్కర్లేదు — ఎప్పుడూ valid object
console.log(findUser(1).getName()); // Yaswanth
console.log(findUser(99).getName()); // Guest (null error రాదు!)
console.log(findUser(99).isGuest()); // true
```

Key Points

- `null` / `undefined` checks తగ్గుతాయి → NullPointerException-లాంటి bugs తగ్గుతాయి
- Default/neutral behaviour ని encapsulate చేస్తుంది
- Logging (NoOpLogger), permissions (GuestUser), strategies

---

43. Concurrency మరియు Thread-Safety

వివరణ

JavaScript **single-threaded** (event loop) - కాబట్టి Java-లాంటి true data races లేవు. కానీ **async operations మధ్య interleaving** వల్ల logical race conditions వస్తాయి. Critical sections ని serialize చేయాలి.

Real-life Scenario

> *ఒకే bank account నుండి రెండు ATMs ఒకేసారి withdraw చేస్తే - రెండూ balance చదివి, రెండూ తీస్తే overdraw అవుతుంది. Lock (mutex) తో ఒక్కసారి ఒకరే access చేసేలా చేయాలి.*

Code - Async Mutex

```
// Async Mutex — ఒక్కో సారి ఒక్క async task మాత్రమే critical section లో
class Mutex {
  #chain = Promise.resolve();
  runExclusive(task) {
    const result = this.#chain.then(() => task());
    this.#chain = result.catch(() => {}); // error వచ్చినా గొలుసు తెగకూడదు
    return result;
  }
}

class BankAccount {
```

```
  #balance = 100;
  #mutex = new Mutex();

  async withdraw(amount) {
    return this.#mutex.runExclusive(async () => {
      const current = this.#balance; // read
      await new Promise((r) => setTimeout(r, 5)); // async gap (race window)
      if (current >= amount) {
        this.#balance = current - amount; // write
        console.log(`✅ Rs.${amount} withdrawn. Balance: ${this.#balance}`);
      } else {
        console.log(`❌ Rs.${amount} denied. Balance: ${this.#balance}`);
      }
    });
  }
}

const acc = new BankAccount();
// రెండు concurrent withdraws - mutex లేకపోతే రెండూ 100 చదివి overdraw అయ్యేవి
await Promise.all([acc.withdraw(70), acc.withdraw(70)]);
// ✅ Rs.70 withdrawn. Balance: 30
// ❌ Rs.70 denied. Balance: 30   (mutex వల్ల serialize అయ్యాయి)
```

Thread-Safe Singleton గమనిక

భాష	Singleton thread-safety
Java/C++	Double-checked locking / synchronized అవసరం
JavaScript	Module caching + single thread → automatic safe

> ***గుర్తుంచుకో:*** *JS లో "concurrency" = async coordination. Mutex, semaphore, queue తో shared state ని కాపాడు. (Producer-Consumer కి Topic 36 Notification / Observer చూడు.)*

---

44. Architecture Patterns (MVC / Layered / Pub-Sub)

వివరణ

Class-level patterns పైన, application ని **layers** గా విడదీసే architectural patterns. LLD interview లో "ఎలా organize చేస్తావు?" అనేదానికి ఇవి సమాధానం.

MVC / MVP / MVVM

Pattern	విభజన	ఎక్కడ
MVC	Model (data) / View (UI) / Controller (logic)	Web frameworks (Rails, Spring)
MVP	Model / View / Presenter (View passive)	Android (పాతది)
MVVM	Model / View / ViewModel (data binding)	Angular, Vue, WPF

> ***ఉమ్మడి ఆలోచన:*** *UI ని business logic నుండి, logic ని data నుండి విడదీయడం (SRP at app scale). Layered architecture: Presentation → Service → Repository → DB.*

Pub-Sub (Event Bus) - Observer యొక్క Architectural రూపం

```javascript
// Event Bus — publishers & subscribers ఒకరినొకరు తెలియకుండా events ద్వారా మాట్లాడతారు
class EventBus {
  #handlers = new Map();
  on(event, handler) {
    if (!this.#handlers.has(event)) this.#handlers.set(event, []);
    this.#handlers.get(event).push(handler);
    return this;
  }
  emit(event, payload) {
    (this.#handlers.get(event) || []).forEach((h) => h(payload));
  }
}

const bus = new EventBus();
// వేర్వేరు modules — ఒకరికొకరు తెలియదు, event ద్వారా మాత్రమే coupled
bus.on("order.placed", (o) => console.log(`📦 Inventory: ${o.item} తగ్గించు`));
bus.on("order.placed", (o) => console.log(`📧 Email: ${o.item} confirm`));
bus.emit("order.placed", { item: "Book" });
// 📦 Inventory: Book తగ్గించు
// 📧 Email: Book confirm
```

Key Points

- MVC/MVP/MVVM = presentation ని logic/data నుండి విడదీయడం (app-scale SRP)
- Layered architecture = Presentation → Service → Repository → DB
- Pub-Sub / Event Bus = Observer ని modules మధ్య scale చేసిన రూపం (microservices, event-driven)

Part 8 — Interview & Reference

---

45. ఏ Pattern ఎప్పుడు వాడాలి? (Cheat Sheet)

సమస్య → Pattern

నీ సమస్య (ఇలా అనిపిస్తే...)	వాడాల్సిన Pattern
ఒకటే object మొత్తం app కి కావాలి	**Singleton**
new చిమ్ముతోంది, ఏ class అనేది runtime లో	**Factory Method**
సంబంధిత objects family match అవ్వాలి	**Abstract Factory**
Constructor లో చాలా (optional) parameters	**Builder**
Object creation ఖరీదు, copy చేయాలి	**Prototype**
రెండు incompatible interfaces కలపాలి	**Adapter**
Abstraction × Implementation రెండూ మారుతున్నాయి	**Bridge**

Tree structure - part & whole ఒకేలా	**Composite**
Runtime లో పొరలుగా behaviour జోడించాలి	**Decorator**
సంక్లిష్ట subsystem కి సులభ ముఖద్వారం	**Facade**
వేలాది సారూప్య objects - memory ఆదా	**Flyweight**
Access control, lazy load, caching	**Proxy**
Request ని handlers గొలుసు గుండా	**Chain of Responsibility**
Action = object (undo/queue/log)	**Command**
Collection ని internal దాచి traverse	**Iterator**
Many-to-many communication గజిబిజి	**Mediator**
State save/restore (undo)	**Memento**
State మారితే అందరికీ notify	**Observer**
Object behaviour దాని state బట్టి మారాలి	**State**
ఒకే పనికి swappable algorithms	**Strategy**
Algorithm skeleton fix, steps మార్చాలి	**Template Method**
Structure మార్చకుండా కొత్త operations	**Visitor**
ఒక చిన్న భాష / expression ని evaluate చేయాలి	**Interpreter**

Beyond GoF (Practical Patterns)

నీ సమస్య	వాడాల్సినది
Dependency ని బయట నుండి inject చేయాలి	**Dependency Injection**
ఖరీదైన objects reuse చేయాలి	**Object Pool**
null కి బదులు do-nothing object	**Null Object**
Modules ని events తో decouple చేయాలి	**Pub-Sub / Event Bus**
Shared state ని async నుండి కాపాడాలి	**Mutex / Lock**

"if/else పెరుగుతోంది" → ఏ pattern?

ఏం మారుతోంది	Pattern
Algorithm/logic	Strategy
Object యొక్క state	State
ఏ object create అవ్వాలి	Factory
ఎవరు request handle చేస్తారు	Chain of Responsibility

> ***బంగారు నియమం:*** *Pattern కోసం సమస్య వెతకకు. సమస్య వచ్చినప్పుడు సరిపడే pattern వాడు. (YAGNI + KISS)*

---

46. LLD Interview Framework (5 అడుగులు)

వివరణ

Interview లో "Design X" అడిగినప్పుడు panic అవ్వకుండా, ఈ 5 అడుగులు follow చెయ్యి. మాట్లాడుతూ (think aloud) చెయ్యడం ముఖ్యం.

5 అడుగులు

```
1. CLARIFY (2-3 నిమిషాలు)
   - Scope ఏమిటి? Users ఎంతమంది? ఏ features తప్పనిసరి?
   - "Parking lot - bikes కూడానా? Payment ఉందా? Multiple floors?"

2. ENTITIES + APIs (Nouns → classes)
   - ముఖ్య classes list చెయ్యి
   - Public methods (APIs) ఏమిటో చెప్పు

3. CLASS DIAGRAM (గీయి)
   - Classes, attributes, methods
   - Relationships (has-a / is-a / uses-a) బాణాలతో

4. PATTERNS + DESIGN
   - ఎక్కడ ఏ pattern? ఎందుకు?
   - SOLID చెక్: ఈ design extensible నా?

5. CODE + EDGE CASES
   - ముఖ్య classes code
   - Edge cases: full, empty, concurrent, invalid input
```

చెప్పకూడని / చెప్పాల్సిన మాటలు

❌ చెప్పకు	✅ చెప్పు
నేరుగా code మొదలుపెట్టడం	"ముందు requirements clarify చేస్తాను"
"ఇలా చేస్తా" (silent)	Think aloud - ఆలోచన చెప్పు
ఒకే perfect solution వెతకడం	Trade-offs మాట్లాడు
అన్ని patterns కూరడం	అవసరమైనవి మాత్రమే (YAGNI)

Trade-offs మాట్లాడటం (ముఖ్యం)

> *"ఇక్కడ Strategy వాడాను ఎందుకంటే pricing rules తరచూ మారతాయి. Inheritance వాడితే class explosion వచ్చేది. కానీ Strategy వల్ల classes కొంచెం పెరుగుతాయి - అది acceptable trade-off."*

Interviewer perfect design కంటే **నీ ఆలోచనా విధానం (reasoning)** ని చూస్తాడు.

---

47. Memory Tips Table - మర్చిపోకూడదంటే

Design Principles

Principle	ఒక్క ముక్కలో
SRP	ఒక class = ఒక పని
OCP	Extension కి open, modification కి closed
LSP	Child, parent స్థానంలో సరిపోవాలి
ISP	చిన్న interfaces, పెద్దవి కాదు
DIP	Abstraction మీద depend, concrete మీద కాదు
DRY	Repeat చేయకు
KISS	Simple గా ఉంచు
YAGNI	అవసరం లేనిది రాయకు
LoD	Friends తోనే మాట్లాడు (a.b.c.d వద్దు)

Design Patterns (గుర్తుంచుకునే analogy)

Pattern	గుర్తుంచుకోవడానికి
Singleton	ఒకే CEO
Factory	Restaurant kitchen (order → dish)
Abstract Factory	Furniture showroom (matching family)
Builder	Subway sandwich (step by step)
Prototype	Resume template copy
Adapter	Travel plug adapter
Bridge	Remote × Device (mix & match)
Composite	Folder లో folder (tree)
Decorator	Coffee + milk + sugar (పొరలు)
Facade	Car start button (లోపలి 10 steps దాచి)
Flyweight	అడవిలో చెట్లు (type share)
Proxy	Credit card (account కి ప్రతినిధి)
Chain of Resp.	Support escalation (L1→Manager→Director)
Command	Order slip (undo/queue)
Iterator	Remote channel up
Mediator	ATC tower (విమానాలు నేరుగా మాట్లాడవు)

Memento	Game save point
Observer	YouTube subscribe
State	Traffic signal (Red→Green→Yellow)
Strategy	Maps route (car/walk/bus)
Template Method	Tea/Coffee recipe skeleton
Visitor	Tax auditor (ఇంటింటికీ visit)
Interpreter	Calculator ("5 + 3" → 8)

Beyond GoF (గుర్తుంచుకునే analogy)

Pattern	గుర్తుంచుకోవడానికి
Dependency Injection	కారులో engine బయట అమర్చడం
Object Pool	Cricket నెట్స్ ball bucket (reuse)
Null Object	Guest account (null కాదు)
Pub-Sub / Event Bus	Radio station (broadcast)

3 Categories గుర్తుంచుకో (23 GoF)

Category	ఏం చేస్తాయి	ఆలోచన
Creational (5)	Objects పుట్టడం	"ఎలా create చేయాలి?"
Structural (7)	Objects కలవడం	"ఎలా కలపాలి?"
Behavioral (11)	Objects మాట్లాడటం	"ఎలా communicate?"

> ***మొత్తం 23 GoF patterns*** *(5+7+11) +* ***Beyond GoF*** *practical patterns (DI, Object Pool, Null Object, Pub-Sub) - ఈ document లో అన్నీ cover అయ్యాయి.*

ముగింపు

> ***గుర్తుంచుకో:***
>
> - *LLD = OOP tools ని ఎప్పుడు, ఎలా వాడాలో నేర్పే craft.*
> - *Principles (SOLID/DRY/KISS/YAGNI) = దిక్సూచి. Patterns = నిరూపితమైన పరిష్కారాలు.*
> - *Pattern కోసం సమస్య వెతకకు; సమస్యకి సరిపడే pattern వాడు.*
> - *ప్రతి pattern కి real-life analogy గుర్తుపెట్టుకో - అప్పుడు మర్చిపోలేవు!*

ఈ document + `OOPS_Telugu.md` కలిస్తే - **OOP పునాది నుండి LLD నైపుణ్యం వరకు** పూర్తి తెలుగు reference.

JavaScript LLD - Complete Telugu Guideప్రతి code snippet Node.js లో run చేసి verify చేయబడింది ✅